

众云信息科技有限公司维修回收综合服务平台V1.0

源程序代码清单（前30页 + 后30页，共60页）

// backend/app/controller/AgentController.php

```
<?php
```

```
namespace app\controller;
```

```
use app\service\AgentService;
```

```
use think\facade\Log;
```

```
class AgentController extends BaseController
```

```
{
```

```
    private const MAX_MESSAGE_LENGTH = 4000;
```

```
    private const MAX_HISTORY_MESSAGES = 12;
```

```
    private const MAX_HISTORY_ITEM_LENGTH = 2000;
```

```
    public function chat()
```

```
    {
```

```
        try {
```

```
            $data = $this->getRequestData();
```

```
            $message = trim((string) ($data['message'] ?? ''));
```

```
            $history = $this->normalizeHistory($data['history'] ?? []);
```

```
            if ($message === '') {
```

```
                return $this->error('消息内容不能为空', 400);
```

```
            }
```

```
            if (mb_strlen($message) > self::MAX_MESSAGE_LENGTH) {
```

```
                return $this->error('消息内容过长，请控制在 4000 个字符以内', 422);
```

```
            }
```

```
            $service = new AgentService();
```

```
            $result = $service->chat($message, $history, $this->getUserId());
```

```
            return $this->success($result);
```

```
        } catch (\Throwable $e) {
```

```
            Log::error('Agent 对话失败: ' . $e->getMessage());
```

```
            return $this->error('Agent 回复失败: ' . $e->getMessage(), 500);
```

```
        }
```

```
    }
```

```
    private function normalizeHistory($history): array
```

```
    {
```

```
        if (!is_array($history)) {
```

```
            return [];
```

```
        }
```

```
        $normalized = [];
```

```
        $history = array_slice($history, -self::MAX_HISTORY_MESSAGES);
```

```
        foreach ($history as $item) {
```

```
            if (!is_array($item)) {
```

```
                continue;
```

```
            }
```

```
            $role = trim((string) ($item['role'] ?? ''));
```

```
            $content = trim((string) ($item['content'] ?? ''));
```

```

        if (!in_array($role, ['user', 'assistant'], true) || $content === '') {
            continue;
        }

        $normalized[] = [
            'role' => $role,
            'content' => mb_substr($content, 0, self::MAX_HISTORY_ITEM_LENGTH),
        ];
    }

    return $normalized;
}
}

```

// backend/app/controller/AgreementController.php

```
<?php
```

```
namespace app\controller;
```

```
use app\model\Agreement;
use app\common\Result;
use think\facade\Db;
```

```
/**
 * 免责协议管理控制器
 */
```

```
class AgreementController
```

```
{
    protected function ensureAgreementsTable()
    {
        try {
            Db::query('SELECT 1 FROM `agreements` LIMIT 1');
            return;
        } catch (\Exception $e) {
        }
    }
}
```

```

    Db::execute("CREATE TABLE IF NOT EXISTS `agreements` (
        `id` int(11) unsigned NOT NULL AUTO_INCREMENT COMMENT '主键 ID',
        `title` varchar(200) NOT NULL COMMENT '协议标题',
        `code` varchar(50) NOT NULL COMMENT '协议编码',
        `content` text COMMENT '协议内容 (HTML)',
        `version` varchar(20) DEFAULT '1.0' COMMENT '版本号',
        `status` tinyint(1) DEFAULT 1 COMMENT '状态: 1 启用 0 禁用',
        `effective_date` date DEFAULT NULL COMMENT '生效日期',
        `remark` text COMMENT '备注',
        `created_by` int(11) DEFAULT NULL COMMENT '创建人 ID',
        `updated_by` int(11) DEFAULT NULL COMMENT '更新人 ID',
        `created_at` datetime DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
        `updated_at` datetime DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP COMMENT '
更新时间',
        PRIMARY KEY (`id`),
        UNIQUE KEY `uk_code` (`code`)
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='免责协议表';");

```

```

    Db::execute("INSERT INTO `agreements` (`title`, `code`, `content`, `version`, `status`, `effective_date`, `remark`)
    VALUES ('设备维修服务免责声明', 'repair_disclaimer',
        '<h2>设备维修服务免责声明</h2><h3>一、服务范围</h3><p>1. 本协议适用于本公司提供的所有设备维修服务。</p><p>2. 服务范围包括但不限于故障诊断、维修、保养、更换配件等。</p><h3>二、免责条款</h3><p>1. 因不可抗力因素（如自然灾害、战争、政府行为等）导致的设备损坏，本公司不承担责任。</p><p>2. 客户未按设备使用说明书操作导致的损坏，本公司不承担责任。</p><p>3. 维修过程中因设备自身老化、磨损等原因导致的二次损坏，本公司不

```

承担责任。

三、维修保证

1. 本公司对维修部位提供 30 天的质量保证期。

2. 质量保证期内，因维修质量问题导致的故障，本公司免费重新维修。

四、其他

1. 本协议自客户签字确认之日起生效。

2. 本协议的最终解释权归本公司所有。

'1.0', 1, '2024-01-01', '设备维修服务标准免责协议')
ON DUPLICATE KEY UPDATE title=VALUES(title);

}

/**

* 获取协议信息

* GET /business/agreement

*/

public function index()

{

try {

\$this->ensureAgreementsTable();

// 获取启用的协议

\$agreement = Agreement::where('status', 1)->order('id', 'desc')->find();

if (!\$agreement) {

// 如果没有启用的协议，返回最新的协议

\$agreement = Agreement::order('id', 'desc')->find();

}

if (!\$agreement) {

return Result::success([

'title' => '设备维修服务免责协议',

'content' => '',

'version' => '1.0',

'status' => 0,

'effective_date' => null,

'updated_at' => null

]);

}

return Result::success([

'id' => \$agreement->id,

'title' => \$agreement->title,

'code' => \$agreement->code,

'content' => \$agreement->content,

'version' => \$agreement->version,

'status' => \$agreement->status,

'effective_date' => \$agreement->effective_date,

'updated_at' => \$agreement->updated_at,

'updater' => \$this->getUpdaterName(\$agreement->updated_by)

]);

} catch (\Exception \$e) {

return Result::error(\$e->getMessage(), 500);

}

}

/**

* 获取协议详情

* GET /business/agreement/:id

*/

public function read(\$id)

{

try {

\$this->ensureAgreementsTable();

\$agreement = Agreement::find(\$id);

if (!\$agreement) {

return Result::error('协议不存在', 404);

}

```

        return Result::success([
            'id' => $agreement->id,
            'title' => $agreement->title,
            'code' => $agreement->code,
            'content' => $agreement->content,
            'version' => $agreement->version,
            'status' => $agreement->status,
            'effective_date' => $agreement->effective_date,
            'remark' => $agreement->remark,
            'created_at' => $agreement->created_at,
            'updated_at' => $agreement->updated_at,
            'updater' => $this->getUpdaterName($agreement->updated_by)
        ]);
    } catch (\Exception $e) {
        return Result::error($e->getMessage(), 500);
    }
}

/**
 * 保存协议
 * POST /business/agreement
 */
public function save()
{
    $data = request()->post();

    try {
        $this->ensureAgreementsTable();
        // 验证
        validate([
            'title' => 'require|max:200',
            'code' => 'require|max:50',
            'content' => 'require'
        ])->check($data);

        // 检查编码是否已存在
        $exists = Agreement::where('code', $data['code'])->find();
        if ($exists) {
            return Result::error('协议编码已存在', 422);
        }

        $agreement = Agreement::create([
            'title' => $data['title'],
            'code' => $data['code'],
            'content' => $data['content'],
            'version' => $data['version'] ?? '1.0',
            'status' => $data['status'] ?? 1,
            'effective_date' => $data['effective_date'] ?? null,
            'remark' => $data['remark'] ?? null,
            'created_by' => $this->getCurrentUserId(),
            'updated_by' => $this->getCurrentUserId()
        ]);

        return Result::success($agreement, '协议创建成功', 201);
    } catch (\think\exception\ValidateException $e) {
        return Result::error($e->getError(), 422);
    } catch (\Exception $e) {
        return Result::error($e->getMessage(), 500);
    }
}

/**

```

```
* 更新协议
* PUT /business/agreement/:id
*/
public function update($id)
{
    $data = request()->put();

    try {
        $this->ensureAgreementsTable();
        $agreement = Agreement::find($id);

        if (!$agreement) {
            return Result::error('协议不存在', 404);
        }

        // 验证
        validate([
            'title' => 'require|max:200',
            'content' => 'require'
        ])->check($data);

        $agreement->save([
            'title' => $data['title'] ?? $agreement->title,
            'content' => $data['content'] ?? $agreement->content,
            'version' => $data['version'] ?? $agreement->version,
            'status' => $data['status'] ?? $agreement->status,
            'effective_date' => $data['effective_date'] ?? $agreement->effective_date,
            'remark' => $data['remark'] ?? $agreement->remark,
            'updated_by' => $this->getCurrentUserId()
        ]);

        return Result::success($agreement, '协议更新成功');
    } catch (\think\Exception\ValidateException $e) {
        return Result::error($e->getError(), 422);
    } catch (\Exception $e) {
        return Result::error($e->getMessage(), 500);
    }
}

/**
* 预览协议
* GET /business/agreement/:id/preview
*/
public function preview($id)
{
    try {
        $this->ensureAgreementsTable();
        $agreement = Agreement::find($id);

        if (!$agreement) {
            return Result::error('协议不存在', 404);
        }

        return Result::success([
            'title' => $agreement->title,
            'content' => $agreement->content,
            'version' => $agreement->version,
            'effective_date' => $agreement->effective_date
        ]);
    } catch (\Exception $e) {
        return Result::error($e->getMessage(), 500);
    }
}
```

```

/**
 * 获取当前用户 ID
 */
protected function getCurrentUserId()
{
    $user = request()->user ?? null;
    return $user ? $user['id'] : 1;
}

/**
 * 获取更新人名称
 */
protected function getUpdaterName($userId)
{
    if (!$userId) {
        return "";
    }

    $user = \app\model\User::find($userId);
    return $user ? ($user->real_name ?: $user->username) : "";
}
}

```

// backend/app/controller/AlipayTestController.php

```
<?php
```

```
namespace app\controller;
```

```
use app\common\Result;
```

```
use app\model\OnlinePayment;
```

```
class AlipayTestController extends BaseController
```

```

{
    // 沙箱环境配置 (从 .env 读取)
    private function getAlipayConfig()
    {
        // 优先从单独文件读取密钥; 文件为空时回退到 .env 多行值
        $privateKey = $this->readKeyFromFile('.alipay_private.pem');
        if ($privateKey === "") {
            $privateKey = $this->loadEnvFromDocker('ALIPAY_PRIVATE_KEY', "");
        }

        $publicKey = $this->readKeyFromFile('.alipay_public.pem');
        if ($publicKey === "") {
            $publicKey = $this->loadEnvFromDocker('ALIPAY_PUBLIC_KEY', "");
        }

        return [
            'app_id' => $this->loadEnvFromDocker('ALIPAY_APP_ID', env('ALIPAY_APP_ID', '')),
            'private_key' => $privateKey,
            'alipay_public_key' => $publicKey,
            'gateway' => $this->loadEnvFromDocker('ALIPAY_GATEWAY', env('ALIPAY_GATEWAY', 'https://openapi-sandbox.dl.alipaydev.com/gateway.do')),
            'notify_url' => $this->loadEnvFromDocker('ALIPAY_NOTIFY_URL', env('ALIPAY_NOTIFY_URL', '')),
            'return_url' => $this->loadEnvFromDocker('ALIPAY_RETURN_URL', env('ALIPAY_RETURN_URL', '')),
            'sandbox' => filter_var($this->loadEnvFromDocker('ALIPAY_SANDBOX', env('ALIPAY_SANDBOX', true)), FILTER_VALIDATE_BOOLEAN),
            // 模拟模式: true=不调用真实API, false=调用真实支付宝

```

```

        'mock_mode' => filter_var($this->loadEnvFromDocker('ALIPAY MOCK_MODE', env('ALIPAY MOCK_MODE
', true)), FILTER_VALIDATE_BOOLEAN),
    ];
}

private function readKeyFromFile($filename)
{
    $filepath = root_path() . $filename;
    if (!file_exists($filepath)) {
        return "";
    }
    $content = file_get_contents($filepath);
    return trim((string) $content);
}

/**
 * 从Docker环境变量或.env文件中加载配置（支持多行值）
 */
private function loadEnvFromDocker($key, $default = "")
{
    // 首先尝试从Docker环境变量获取
    $value = getenv($key);

    if ($value === false) {
        // 如果环境变量不存在，直接从.env文件读取
        return $this->readKeyFromEnvFile($key, $default);
    }

    // 检查是否是转义的多行字符串（Docker env_file处理方式）
    if (strpos($value, '\\n') !== false) {
        $value = str_replace("\\n", "\n", $value);
    }

    // 如果值太短或明显不完整，尝试从文件读取
    if (strpos($value, '-----BEGIN') !== false && strpos($value, '-----END') === false) {
        return $this->readKeyFromEnvFile($key, $default);
    }

    return $value;
}

/**
 * 直接从.env文件读取多行键值
 */
private function readKeyFromEnvFile($key, $default = "")
{
    $envFile = root_path() . '.env';

    if (!file_exists($envFile)) {
        return $default;
    }

    // 使用file_get_contents读取完整内容，然后解析
    $content = file_get_contents($envFile);

    // 查找键的位置
    $pattern = '/^' . preg_quote($key, '/') . '\s*=\s*(.*)$/ms';
    if (!preg_match($pattern, $content, $matches)) {
        return $default;
    }

    $value = $matches[1];

```

```

// 检查是否是多行PEM格式
if (strpos($value, '-----BEGIN') !== false) {
    // 提取完整的PEM块
    $beginPos = strpos($value, '-----BEGIN');
    $endMarker = strpos($value, '-----END', $beginPos);

    if ($endMarker !== false) {
        // 查找END行的结束位置
        $endLinePos = strpos($content, "\n", strpos($content, '-----END', strpos($content, $key)));

        if ($endLinePos === false) {
            $endLinePos = strlen($content);
        }
    }

    // 从原始内容中提取完整的PEM块
    $keyStart = strpos($content, '-----BEGIN', strpos($content, $key));
    $keyEnd = strpos($content, '-----END', $keyStart) + 15; // 15是"-----END RSA PRIVATE KEY-----"的长度

    if ($keyEnd > $keyStart) {
        $fullKey = substr($content, $keyStart, $keyEnd - $keyStart);
        return $fullKey;
    }
}

return $value;
}

// 沙箱默认密钥（支付宝沙箱官方提供的测试账号）
private function getSandboxDefaultKeys()
{
    return [
        'app_id' => '2021000122600000', // 沙箱测试AppID（示例）
        'private_key' => '',
        'alipay_public_key' => '',
    ];
}

public function create()
{
    try {
        $data = $this->getRequestData([
            'subject' => 'require',
            'total_amount' => 'require|float|gt:0',
            'out_trade_no' => 'require',
        ]);

        $subject = trim((string) $data['subject']);
        $body = trim((string) ($data['body'] ?? ''));
        $outTradeNo = trim((string) $data['out_trade_no']);
        $totalAmount = number_format((float) $data['total_amount'], 2, '.', '');

        $config = $this->getAlipayConfig();
        $appId = $config['app_id'];
        $privateKey = $config['private_key'];

        // 创建或更新本地获取支付记录
        $payment = OnlinePayment::where('order_no', $outTradeNo)->find();

        if (!$payment) {
            $payment = new OnlinePayment();
            $payment->order_no = $outTradeNo;
        }
    }
}

```

```
$payment->amount = $totalAmount;
$payment->payment_method = 'alipay';
$payment->remark = $subject . ($body ? ' - ' . $body : '');

// 如果订单不是已支付/已退款状态, 则设为待支付
if ($payment->status !== 'paid' && $payment->status !== 'refunded') {
    $payment->status = 'pending';
}

$payment->save();

// 模拟模式: 不需要真实密钥
if ($config['mock_mode']) {
    $mockPayUrl = $this->buildMockCashierUrl($outTradeNo);
    return Result::success([
        'pay_url' => $mockPayUrl,
        'qr_code' => null,
        'out_trade_no' => $outTradeNo,
        'total_amount' => $totalAmount,
        'mock_mode' => true,
        'mock_test_info' => [
            'sandbox_account' => 'sandboxbt01@sandbox.com',
            'sandbox_password' => '111111',
            'note' => '沙箱环境已下线, 此为本地模拟测试',
            'instruction' => '点击“打开支付页面”进入模拟支付宝收银台, 完成支付或取消订单'
        ],
        'message' => '支付订单创建成功 (模拟模式)'
    ], '支付订单创建成功 (模拟模式)');
}

// 真实沙箱模式: 需要配置密钥
if (empty($appld) || empty($privateKey)) {
    return Result::error('请先配置支付宝密钥: ALIPAY_APP_ID 和 ALIPAY_PRIVATE_KEY', 500);
}

// 构建支付宝支付参数
$bizContent = [
    'out_trade_no' => $outTradeNo,
    'total_amount' => $totalAmount,
    'subject' => $subject,
    'body' => $body,
    'product_code' => 'FAST_INSTANT_TRADE_PAY',
];

$params = [
    'app_id' => $appld,
    'method' => 'alipay.trade.page.pay',
    'format' => 'JSON',
    'charset' => 'utf-8',
    'sign_type' => 'RSA2',
    'timestamp' => date('Y-m-d H:i:s'),
    'version' => '1.0',
    'notify_url' => $config['notify_url'],
    'return_url' => $config['return_url'],
    'biz_content' => json_encode($bizContent, JSON_UNESCAPED_UNICODE),
];

// 生成签名
$params['sign'] = $this->generateSign($params, $privateKey);

// 构建支付 URL
$payUrl = $config['gateway'] . '?' . http_build_query($params);
```

```

        return Result::success([
            'pay_url' => $payUrl,
            'qr_code' => null,
            'out_trade_no' => $outTradeNo,
            'total_amount' => $totalAmount,
            'mock_mode' => false,
        ], '支付订单创建成功');
    } catch (\Throwable $e) {
        return Result::error('创建支付订单失败: ' . $e->getMessage(), 500);
    }
}

/**
 * 模拟支付宝收银台页面
 * GET /api/payment/alipay/mock-page
 */
public function mockPage()
{
    $outTradeNo = trim((string) request()->get('out_trade_no', ''));

    if ($outTradeNo === '') {
        return response('<h3>缺少订单编号</h3>', 400, ['Content-Type' => 'text/html; charset=UTF-8']);
    }

    $payment = OnlinePayment::where('order_no', $outTradeNo)->find();
    if (!$payment) {
        return response('<h3>订单不存在</h3>', 404, ['Content-Type' => 'text/html; charset=UTF-8']);
    }

    $statusMap = [
        'pending' => '等待买家付款',
        'paid' => '支付成功',
        'cancelled' => '交易关闭',
        'refunded' => '已退款',
    ];

    $statusText = $statusMap[$payment->status] ?? '等待买家付款';
    $amount = number_format((float) $payment->amount, 2, '.', '');
    $subject = htmlspecialchars($payment->remark ? '支付宝测试订单', ENT_QUOTES, 'UTF-8');
    $orderNo = htmlspecialchars($payment->order_no, ENT_QUOTES, 'UTF-8');
    $status = htmlspecialchars($payment->status, ENT_QUOTES, 'UTF-8');
    $statusText = htmlspecialchars($statusText, ENT_QUOTES, 'UTF-8');

    $html = <<<HTML
<!DOCTYPE html>
<html lang="zh-CN">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>支付宝模拟收银台</title>
<style>
* { box-sizing: border-box; }
body {
    margin: 0;
    font-family: "Microsoft YaHei", sans-serif;
    background: linear-gradient(180deg, #0d6efd 0, #f5f7fb 220px);
    color: #1f2d3d;
}
.wrap {
    max-width: 760px;
    margin: 0 auto;
    padding: 40px 20px 60px;

```

```
}
.brand {
  color: #fff;
  font-size: 28px;
  font-weight: 700;
  letter-spacing: 1px;
  margin-bottom: 20px;
}
.card {
  background: #fff;
  border-radius: 18px;
  box-shadow: 0 20px 50px rgba(16, 47, 92, 0.18);
  overflow: hidden;
}
.card-header {
  padding: 24px 30px;
  border-bottom: 1px solid #eef2f7;
  display: flex;
  justify-content: space-between;
  align-items: center;
}
.title {
  font-size: 22px;
  font-weight: 700;
  color: #1677ff;
}
.badge {
  background: #ecf5ff;
  color: #1677ff;
  padding: 6px 12px;
  border-radius: 999px;
  font-size: 13px;
}
.content {
  padding: 30px;
}
.amount {
  text-align: center;
  margin-bottom: 28px;
}
.amount .label {
  color: #8a94a6;
  font-size: 14px;
}
.amount .value {
  margin-top: 8px;
  font-size: 42px;
  font-weight: 700;
  color: #111827;
}
.meta {
  background: #f8fafc;
  border-radius: 14px;
  padding: 18px 20px;
  margin-bottom: 24px;
}
.row {
  display: flex;
  justify-content: space-between;
  gap: 16px;
  padding: 10px 0;
  border-bottom: 1px dashed #dbe3ef;
}
```

```
.row:last-child { border-bottom: 0; }
.label { color: #6b7280; }
.value { color: #111827; text-align: right; word-break: break-all; }
.tips {
  padding: 16px 18px;
  border-radius: 12px;
  background: #fff7e6;
  color: #8a5a00;
  line-height: 1.7;
  margin-bottom: 24px;
  font-size: 14px;
}
.actions {
  display: flex;
  gap: 14px;
  flex-wrap: wrap;
}
button {
  border: 0;
  border-radius: 12px;
  padding: 14px 22px;
  font-size: 16px;
  cursor: pointer;
  transition: transform .15s ease, box-shadow .15s ease, opacity .15s ease;
}
button:hover { transform: translateY(-1px); }
button:disabled { opacity: .55; cursor: not-allowed; transform: none; }
.btn-primary {
  background: linear-gradient(135deg, #1677ff, #4096ff);
  color: #fff;
  box-shadow: 0 10px 20px rgba(22, 119, 255, .22);
  flex: 1;
}
.btn-secondary {
  background: #f3f4f6;
  color: #374151;
  flex: 1;
}
.btn-link {
  background: transparent;
  color: #1677ff;
  padding-left: 0;
  padding-right: 0;
}
.footer-note {
  margin-top: 18px;
  color: #94a3b8;
  font-size: 13px;
}
.success {
  color: #16a34a;
  font-weight: 700;
}
.closed {
  color: #6b7280;
  font-weight: 700;
}
@media (max-width: 640px) {
  .card-header, .content { padding: 20px; }
  .amount .value { font-size: 34px; }
  .actions { flex-direction: column; }
}
</style>
```

```
</head>
<body>
  <div class="wrap">
    <div class="brand">支付宝</div>
    <div class="card">
      <div class="card-header">
        <div class="title">模拟收银台</div>
        <div class="badge">测试环境</div>
      </div>
      <div class="content">
        <div class="amount">
          <div class="label">应付金额</div>
          <div class="value">¥ {$amount}</div>
        </div>

        <div class="meta">
          <div class="row"><div class="label">订单编号</div><div class="value">{$orderNo}</div></div>
          <div class="row"><div class="label">订单说明</div><div class="value">{$subject}</div></div>
          <div class="row"><div class="label">买家账号</div><div class="value">sandboxbt01@sandbox.com</div>
        </div>

        <div class="row"><div class="label">当前状态</div><div class="value" id="statusText" data-status="{ $sta
tus}">{$statusText}</div></div>
      </div>

      <div class="tips">
        这是项目内置的支付宝模拟支付页面，用于完整演示“创建订单 -> 打开收银台 -> 确认支付/取消 -> 查询结果”的
        测试流程。
      </div>

      <div class="actions">
        <button id="payBtn" class="btn-primary">确认支付</button>
        <button id="cancelBtn" class="btn-secondary">取消订单</button>
      </div>

      <div style="margin-top: 10px;">
        <button id="closeBtn" class="btn-link">关闭当前页面</button>
      </div>

      <div class="footer-note">
        支付成功后，请返回“支付宝测试”页面点击“查询支付状态”或查看历史记录刷新结果。
      </div>
    </div>
  </div>

  <script>
    const orderNo = {$this->jsonEncodeForJs($payment->order_no)};
    const payBtn = document.getElementById('payBtn');
    const cancelBtn = document.getElementById('cancelBtn');
    const closeBtn = document.getElementById('closeBtn');
    const statusTextEl = document.getElementById('statusText');

    const refreshButtonState = () => {
      const status = statusTextEl.dataset.status;
      payBtn.disabled = status === 'paid' || status === 'refunded';
      cancelBtn.disabled = status === 'paid' || status === 'cancelled' || status === 'refunded';
    };

    const updateStatus = (status, text) => {
      statusTextEl.dataset.status = status;
      statusTextEl.textContent = text;
      statusTextEl.className = status === 'paid' ? 'value success' : (status === 'cancelled' ? 'value closed' : 'value');
      refreshButtonState();
    };
  </script>
</body>
</html>
```

```

};

const submitAction = async (url, successStatus, successText) => {
  try {
    const response = await fetch(url, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ out_trade_no: orderNo })
    });
    const result = await response.json();
    if (!response.ok || (result.code !== 200 && result.code !== 0)) {
      throw new Error(result.message || '操作失败');
    }
    updateStatus(successStatus, successText);
    alert(result.message || '操作成功');
  } catch (error) {
    alert(error.message || '操作失败');
  }
};

payBtn.addEventListener('click', () => submitAction('/api/payment/alipay/mock', 'paid', '支付成功'));
cancelBtn.addEventListener('click', () => submitAction('/api/payment/alipay/mock-cancel', 'cancelled', '交易关闭'));
closeBtn.addEventListener('click', () => window.close());

refreshButtonState();
</script>
</body>
</html>
HTML;

return response($html, 200, ['Content-Type' => 'text/html; charset=UTF-8']);
}

public function query()
{
    $outTradeNo = trim((string) request()->get('out_trade_no', ''));

    if ($outTradeNo === '') {
        return Result::error('订单编号不能为空', 422);
    }

    $config = $this->getAlipayConfig();

    // 模拟模式：从本地数据库获取
    if ($config['mock_mode']) {
        $payment = OnlinePayment::where('order_no', $outTradeNo)->find();

        if (!$payment) {
            return Result::error('订单不存在', 404);
        }

        $tradeStatus = match($payment->status) {
            'paid' => 'TRADE_SUCCESS',
            'pending' => 'WAIT_BUYER_PAY',
            'cancelled' => 'TRADE_CLOSED',
            'refunded' => 'TRADE_CLOSED',
            default => 'WAIT_BUYER_PAY',
        };

        return Result::success([
            'out_trade_no' => $payment->order_no,
            'trade_no' => $payment->trade_no,
        ]);
    }
}

```

```

        'subject' => $payment->remark ? '支付宝测试订单',
        'total_amount' => $payment->amount,
        'buyer_logon_id' => 'sandboxbt01@sandbox.com',
        'trade_status' => $tradeStatus,
        'gmt_payment' => $payment->paid_at ? date('Y-m-d H:i:s', strtotime($payment->paid_at)) : null,
        'refund_amount' => $payment->refund_amount,
        'refund_at' => $payment->refund_at ? date('Y-m-d H:i:s', strtotime($payment->refund_at)) : null,
        'mock_mode' => true,
    ], '查询成功 (模拟数据) ');
}

$appId = $config['app_id'];
$privateKey = $config['private_key'];

if (empty($appId) || empty($privateKey)) {
    return Result::error('请先配置支付宝密钥', 500);
}

// 构建查询请求
$bizContent = [
    'out_trade_no' => $outTradeNo,
];

$params = [
    'app_id' => $appId,
    'method' => 'alipay.trade.query',
    'format' => 'JSON',
    'charset' => 'utf-8',
    'sign_type' => 'RSA2',
    'timestamp' => date('Y-m-d H:i:s'),
    'version' => '1.0',
    'biz_content' => json_encode($bizContent, JSON_UNESCAPED_UNICODE),
];

try {
    $params['sign'] = $this->generateSign($params, $privateKey);

    // 发送请求
    $response = $this->httpPost($config['gateway'], $params);
    $result = json_decode($response, true);

    if (isset($result['alipay_trade_query_response'])) {
        $data = $result['alipay_trade_query_response'];
        return Result::success([
            'out_trade_no' => $data['out_trade_no'] ?? $outTradeNo,
            'trade_no' => $data['trade_no'] ?? null,
            'subject' => $data['subject'] ?? '支付宝测试订单',
            'total_amount' => $data['total_amount'] ?? null,
            'buyer_logon_id' => $data['buyer_logon_id'] ?? null,
            'trade_status' => $data['trade_status'] ?? 'WAIT_BUYER_PAY',
            'gmt_payment' => $data['gmt_payment'] ?? null,
        ], '查询成功');
    }

    return Result::error('支付宝返回数据格式错误', 500);
} catch (\Throwable $e) {
    return Result::error('查询失败: ' . $e->getMessage(), 500);
}
}

/**
 * 模拟支付接口
 * POST /api/payment/alipay/mock

```

```
*/
public function mockPay()
{
    try {
        $data = $this->getRequestData([
            'out_trade_no' => 'require',
        ]);

        $outTradeNo = trim((string) $data['out_trade_no']);
        $config = $this->getAlipayConfig();

        if (!$config['mock_mode']) {
            return Result::error('模拟支付仅在模拟模式下可用', 400);
        }

        // 获取支付记录
        $payment = OnlinePayment::where('order_no', $outTradeNo)->find();

        if (!$payment) {
            return Result::error('订单不存在', 404);
        }

        if ($payment->status === 'paid') {
            return Result::error('该订单已完成支付', 400);
        }

        if ($payment->status === 'refunded') {
            return Result::error('该订单已退款', 400);
        }

        // 生成模拟交易号
        $tradeNo = 'ALI' . date('YmdHis') . substr(md5($outTradeNo), 0, 6);

        // 更新支付状态
        $payment->status = 'paid';
        $payment->trade_no = $tradeNo;
        $payment->paid_at = date('Y-m-d H:i:s');
        $payment->save();

        return Result::success([
            'out_trade_no' => $payment->order_no,
            'trade_no' => $payment->trade_no,
            'total_amount' => $payment->amount,
            'trade_status' => 'TRADE_SUCCESS',
            'gmt_payment' => $payment->paid_at,
            'mock_mode' => true,
        ], '模拟支付成功');
    } catch (\Throwable $e) {
        return Result::error('模拟支付失败: ' . $e->getMessage(), 500);
    }
}

/**
 * 模拟取消支付
 * POST /api/payment/alipay/mock-cancel
 */
public function mockCancel()
{
    try {
        $data = $this->getRequestData([
            'out_trade_no' => 'require',
        ]);
    }
}
```

```
$outTradeNo = trim((string) $data['out_trade_no']);
$config = $this->getAlipayConfig();

if (!$config['mock_mode']) {
    return Result::error('模拟取消仅在模拟模式下可用', 400);
}

// 获取支付记录
$payment = OnlinePayment::where('order_no', $outTradeNo)->find();

if (!$payment) {
    return Result::error('订单不存在', 404);
}

if ($payment->status === 'paid') {
    return Result::error('已支付的订单无法取消', 400);
}

if ($payment->status === 'cancelled') {
    return Result::success(null, '订单已取消');
}

// 更新支付状态
$payment->status = 'cancelled';
$payment->cancelled_at = date('Y-m-d H:i:s');
$payment->save();

return Result::success([
    'out_trade_no' => $payment->order_no,
    'trade_status' => 'TRADE_CLOSED',
    'mock_mode' => true,
], '模拟取消成功');
} catch (\Throwable $e) {
    return Result::error('模拟取消失败: ' . $e->getMessage(), 500);
}
}

/**
 * 生成 RSA2 签名
 */
private function generateSign($params, $privateKey)
{
    // 排序并拼接参数
    ksort($params);
    $signString = '';
    foreach ($params as $key => $value) {
        // biz_content 不参与签名计算
        if ($key !== 'sign' && $key !== 'biz_content' && $value !== '' && $value !== null) {
            $signString .= $key . '=' . $value . '&';
        }
    }
    $signString = rtrim($signString, '&');

    // 处理私钥格式
    $privateKeyPem = $this->formatPrivateKey($privateKey);

    $key = openssl_pkey_get_private($privateKeyPem);
    if (!$key) {
        throw new \Exception('私钥格式错误, 请检查 ALIPAY_PRIVATE_KEY 配置');
    }

    $signature = '';
    openssl_sign($signString, $signature, $key, OPENSSL_ALGO_SHA256);
```

```
        return base64_encode($signature);
    }

    /**
     * 验签方法
     */
    private function verifySign($params, $publicKey)
    {
        // 解析公钥
        $publicKeyPem = $this->formatPublicKey($publicKey);
        $key = openssl_pkey_get_public($publicKeyPem);
        if (!$key) {
            throw new \Exception('公钥格式错误');
        }

        // 从参数中获取签名
        $sign = $params['sign'] ?? '';
        unset($params['sign']);

        // 排序并拼接参数 (biz_content 不参与签名)
        ksort($params);
        $signString = '';
        foreach ($params as $key => $value) {
            if ($key !== 'sign' && $key !== 'biz_content' && $value !== '' && $value !== null) {
                $signString .= $key . '=' . $value . '&';
            }
        }
        $signString = rtrim($signString, '&');

        // 验签
        return openssl_verify($signString, base64_decode($sign), $key, OPENSSL_ALGO_SHA256);
    }

    /**
     * 格式化私钥
     */
    private function formatPrivateKey($privateKey)
    {
        // 如果已经是 PEM 格式, 直接返回
        if (strpos($privateKey, '-----BEGIN') !== false) {
            return $privateKey;
        }

        // 尝试 PKCS1 格式
        $pem = "-----BEGIN RSA PRIVATE KEY-----\n";
        $pem .= chunk_split($privateKey, 64, "\n");
        $pem .= "-----END RSA PRIVATE KEY-----";

        if (openssl_pkey_get_private($pem)) {
            return $pem;
        }

        // 尝试 PKCS8 格式
        $pem = "-----BEGIN PRIVATE KEY-----\n";
        $pem .= chunk_split($privateKey, 64, "\n");
        $pem .= "-----END PRIVATE KEY-----";

        if (openssl_pkey_get_private($pem)) {
            return $pem;
        }

        return $privateKey;
    }
}
```

```
/**
 * 格式化公钥
 */
private function formatPublicKey($publicKey)
{
    // 如果已经是 PEM 格式, 直接返回
    if (strpos($publicKey, '-----BEGIN') !== false) {
        return $publicKey;
    }

    // 添加 PEM 标记
    $pem = "-----BEGIN PUBLIC KEY-----\n";
    $pem .= chunk_split($publicKey, 64, "\n");
    $pem .= "-----END PUBLIC KEY-----";

    return $pem;
}

/**
 * 异步通知处理
 * POST /api/payment/alipay/notify
 */
public function notify()
{
    try {
        $data = request()->post();
        $config = $this->getAlipayConfig();

        // 模拟模式下不做验签
        if ($config['mock_mode']) {
            \think\facade\Log::info('支付宝异步通知 (模拟模式) : ' . json_encode($data));

            return 'success';
        }

        // 验签
        if (!$this->verifySign($data, $config['alipay_public_key'])) {
            \think\facade\Log::error('支付宝异步通知验签失败');
            return 'fail';
        }

        $outTradeNo = $data['out_trade_no'] ?? '';
        $tradeStatus = $data['trade_status'] ?? '';

        // 更新订单状态
        $payment = OnlinePayment::where('order_no', $outTradeNo)->find();
        if ($payment) {
            if ($tradeStatus === 'TRADE_SUCCESS' || $tradeStatus === 'TRADE_FINISHED') {
                $payment->status = 'paid';
                $payment->trade_no = $data['trade_no'] ?? '';
                $payment->paid_at = date('Y-m-d H:i:s');
            } elseif ($tradeStatus === 'TRADE_CLOSED') {
                $payment->status = 'cancelled';
                $payment->cancelled_at = date('Y-m-d H:i:s');
            }
            $payment->save();
        }

        return 'success';
    } catch (\Throwable $e) {
        \think\facade\Log::error('支付宝异步通知处理失败: ' . $e->getMessage());
        return 'fail';
    }
}
```

```
    }
}

/**
 * 模拟退款
 * POST /api/payment/alipay/mock-refund
 */
public function mockRefund()
{
    try {
        $data = $this->getRequestData([
            'out_trade_no' => 'require',
            'refund_amount' => 'float',
        ]);

        $outTradeNo = trim((string) $data['out_trade_no']);
        $refundAmount = isset($data['refund_amount']) ? (float) $data['refund_amount'] : null;
        $config = $this->getAlipayConfig();

        if (!$config['mock_mode']) {
            return Result::error('模拟退款仅在模拟模式下可用', 400);
        }

        // 获取支付记录
        $payment = OnlinePayment::where('order_no', $outTradeNo)->find();

        if (!$payment) {
            return Result::error('订单不存在', 404);
        }

        if ($payment->status !== 'paid') {
            return Result::error('只有已支付的订单才能退款', 400);
        }

        if ($payment->status === 'refunded') {
            return Result::error('该订单已退款', 400);
        }

        if ($refundAmount && $refundAmount > $payment->amount) {
            return Result::error('退款金额不能超过支付金额', 400);
        }

        // 更新退款状态
        $payment->status = 'refunded';
        $payment->refund_amount = $refundAmount ?: $payment->amount;
        $payment->refund_at = date('Y-m-d H:i:s');
        $payment->save();

        return Result::success([
            'out_trade_no' => $payment->order_no,
            'refund_amount' => $payment->refund_amount,
            'mock_mode' => true,
        ], '模拟退款成功');
    } catch (\Throwable $e) {
        return Result::error('模拟退款失败: ' . $e->getMessage(), 500);
    }
}

/**
 * 发送 HTTP POST 请求
 */
private function httpPost($url, $params)
{
```

```

    $ch = curl_init();
    curl_setopt_array($ch, [
        CURLOPT_URL => $url,
        CURLOPT_POST => true,
        CURLOPT_POSTFIELDS => http_build_query($params),
        CURLOPT_RETURNTRANSFER => true,
        CURLOPT_TIMEOUT => 30,
        CURLOPT_SSL_VERIFYPEER => false,
        CURLOPT_SSL_VERIFYHOST => false,
    ]);
    $response = curl_exec($ch);
    $error = curl_error($ch);
    curl_close($ch);

    if ($error) {
        throw new \Exception('HTTP 请求失败: ' . $error);
    }

    return $response;
}

private function buildMockCashierUrl(string $outTradeNo): string
{
    $base = rtrim((string) request()->domain(), '/');
    return $base . '/api/payment/alipay/mock-page?out_trade_no=' . urlencode($outTradeNo);
}

private function jsonEncodeForJs(string $value): string
{
    return json_encode($value, JSON_UNESCAPED_UNICODE | JSON_UNESCAPED_SLASHES);
}
}

```

// backend/app/controller/AttachmentController.php

```
<?php
```

```

namespace app\controller;

use app\common\Result;
use app\service\AttachmentService;
use think\facade\Log;
use think\facade\Response;

class AttachmentController extends BaseController
{
    protected $service;

    public function __construct()
    {
        $this->service = new AttachmentService();
    }

    /**
     * 获取附件列表
     * GET /attachments?target_type=sales&target_id=1
     */
    public function index()
    {
        $targetType = request()->get('target_type', '');
        $targetId = request()->get('target_id', null);
    }
}

```

```
$category = request()->get('category', '');

$list = $this->service->getList($targetType, $targetId ?: null, $category);
return Result::success($list);
}

/**
 * 上传文件
 * POST /attachments/upload
 */
public function upload()
{
    $file = request()->file('file');
    if (!$file) {
        return Result::error('没有文件被上传', 400);
    }

    $targetType = request()->post('target_type', 'general', 'trim');
    $targetId = request()->post('target_id', 0, 'intval');
    $category = request()->post('category', '', 'trim');

    try {
        // 在 move 之前获取文件信息 (move 后临时文件会被删除)
        $fileSize = $file->getSize();
        $originalName = $file->getOriginalName();
        $ext = $file->getOriginalExtension();

        // 验证文件大小
        if ($fileSize > AttachmentService::MAX_UPLOAD_SIZE) {
            return Result::error('文件大小不能超过 30MB', 400);
        }

        // 构建存储路径: uploads/<target_type>/<category>/<filename>
        $relativeDir = $targetType;
        if ($category) {
            $relativeDir .= '/' . $category;
        }
        $targetDir = AttachmentService::UPLOAD_DIR . '/' . $relativeDir;

        // 确保目录存在
        if (!is_dir($targetDir) && !mkdir($targetDir, 0755, true) && !is_dir($targetDir)) {
            throw new \RuntimeException('无法创建上传目录');
        }

        // 生成唯一文件名
        $uniqueName = md5($originalName . uniqid() . time()) . '.' . $ext;
        $relativePath = $relativeDir . '/' . $uniqueName;

        // 移动文件到 Docker 数据卷目录
        $file->move($targetDir, $uniqueName);

        // 保存到数据库
        $attachment = $this->service->create(
            $targetType,
            $targetId,
            $category ?: null,
            $originalName,
            $relativePath,
            $fileSize
        );
    }

    return Result::success([
        'id' => $attachment->id,
    ]
);
```

```

        'file_name' => $originalName,
        'file_path' => $relativePath,
        'url' => '/uploads/' . $relativePath,
        'file_size' => $attachment->file_size,
    ], '上传成功');

} catch (\Exception $e) {
    Log::error('附件上传失败: ' . $e->getMessage());
    return Result::error('上传失败: ' . $e->getMessage(), 500);
}
}

/**
 * 预览/查看文件（浏览器内预览 PDF/图片）
 * GET /attachments/serve/:id
 */
public function serve($id)
{
    try {
        $attachment = $this->service->getById($id);
        $absPath = AttachmentService::UPLOAD_DIR . '/' . ltrim($attachment->file_path, '/');

        if (!is_file($absPath)) {
            return Result::error('文件不存在', 404);
        }

        // 根据扩展名设置 Content-Type
        $ext = strtolower(pathinfo($attachment->file_name, PATHINFO_EXTENSION));
        $mimeMap = [
            'pdf' => 'application/pdf',
            'jpg' => 'image/jpeg',
            'jpeg' => 'image/jpeg',
            'png' => 'image/png',
            'gif' => 'image/gif',
            'webp' => 'image/webp',
            'svg' => 'image/svg+xml',
            'txt' => 'text/plain; charset=utf-8',
        ];

        $contentType = $mimeMap[$ext] ?? 'application/octet-stream';

        return response(file_get_contents($absPath), 200, [
            'Content-Type' => $contentType,
            'Content-Disposition' => 'inline; filename="' . $attachment->file_name . '"',
            'Cache-Control' => 'public, max-age=86400',
        ]);

    } catch (\RuntimeException $e) {
        return Result::error($e->getMessage(), 404);
    } catch (\Exception $e) {
        return Result::error('预览失败: ' . $e->getMessage(), 500);
    }
}

/**
 * 下载文件
 * GET /attachments/download/:id
 */
public function download($id)
{
    try {
        $attachment = $this->service->getById($id);
        $absPath = AttachmentService::UPLOAD_DIR . '/' . ltrim($attachment->file_path, '/');

```

```
        if (!is_file($absPath)) {
            return Result::error('文件不存在', 404);
        }

        return download($absPath, $attachment->file_name);

    } catch (\RuntimeException $e) {
        return Result::error($e->getMessage(), 404);
    } catch (\Exception $e) {
        return Result::error('下载失败: ' . $e->getMessage(), 500);
    }
}

/**
 * 删除附件
 * DELETE /attachments/:id
 */
public function delete($id)
{
    try {
        $this->service->delete($id, true);
        return Result::success(null, '删除成功');
    } catch (\RuntimeException $e) {
        return Result::error($e->getMessage(), 404);
    } catch (\Exception $e) {
        return Result::error('删除失败: ' . $e->getMessage(), 500);
    }
}
}
```

// backend/app/controller/AuthController.php

```
<?php
```

```
namespace app\controller;
```

```
use app\service\JwtService;
use app\validate>LoginValidate;
use app\model\User;
use app\common\Result;
```

```
class AuthController
```

```
{
    /**
     * 用户登录
     * POST /auth/login
     */
    public function login()
    {
        $data = request()->post();
        if (empty($data)) {
            $raw = file_get_contents('php://input');
            $json = json_decode($raw, true);
            if (is_array($json)) {
                $data = $json;
            }
        }

        // 验证输入
        try {
```

```

        validate(LoginValidate::class)
            ->check($data);
    } catch (\think\exception\ValidateException $e) {
        return Result::error($e->getError(), 422);
    }

    // 查找用户
    $user = User::where('username', $data['username'])->find();

    if (!$user) {
        return Result::error('用户名或密码错误', 401);
    }

    // 验证密码
    if (!password_verify($data['password'], $user->password)) {
        return Result::error('用户名或密码错误', 401);
    }

    // 检查用户状态
    if ($user->status != 1) {
        return Result::error('账号已被禁用', 403);
    }

    // 生成 Token
    $accessToken = JwtService::createAccessToken($user->id, $user->role_type);
    $refreshToken = JwtService::createRefreshToken($user->id);

    // 更新最后登录时间
    $user->last_login_at = date('Y-m-d H:i:s');
    $user->save();

    // 返回 Token 和用户信息
    return Result::success([
        'access_token' => $accessToken,
        'refresh_token' => $refreshToken,
        'token_type' => 'Bearer',
        'expires_in' => config('jwt.access_ttl'),
        'user' => [
            'id' => $user->id,
            'username' => $user->username,
            'real_name' => $user->real_name,
            'role_type' => $user->role_type,
            'department_id' => $user->department_id,
        ],
    ], '登录成功');
}

/**
 * 刷新 Token
 * POST /auth/refresh
 */
public function refresh()
{
    $refreshToken = request()->post('refresh_token');

    if (empty($refreshToken)) {
        return Result::error('Refresh Token 不能为空', 400);
    }

    try {
        $newAccessToken = JwtService::refreshAccessToken($refreshToken);

        return Result::success([

```

```

        'access_token' => $newAccessToken,
        'token_type' => 'Bearer',
        'expires_in' => config('jwt.access_ttl'),
    ], 'Token 刷新成功');
    } catch (\Exception $e) {
        return Result::error($e->getMessage(), 401);
    }
}

/**
 * 用户登出
 * POST /auth/logout
 */
public function logout()
{
    $userId = request()->userId;

    // 撤销 Refresh Token
    JwtService::revokeRefreshToken($userId);

    return Result::success(null, '登出成功');
}

/**
 * 获取当前用户信息
 * GET /auth/profile
 */
public function profile()
{
    $userId = request()->userId;

    $user = User::with(['department']->find($userId);

    if (!$user) {
        return Result::error('用户不存在', 404);
    }

    return Result::success([
        'id' => $user->id,
        'username' => $user->username,
        'real_name' => $user->real_name,
        'phone' => $user->phone,
        'email' => $user->email,
        'role_type' => $user->role_type,
        'department_id' => $user->department_id,
        'department_name' => $user->department ? $user->department->name : null,
        'last_login_at' => $user->last_login_at,
    ]);
}
}
}

```

// backend/app/controller/BaseController.php

<?php

namespace app\controller;

use app\common\Result;

use think\facade\Log;

/**

```
* 控制器基类
*/
class BaseController
{
    /**
     * 操作成功返回数据
     * @param mixed $data 返回数据
     * @param string $message 提示信息
     * @param int $code 状态码
     * @return \think\Response
     */
    protected function success($data = null, $message = 'success', $code = 200)
    {
        return Result::success($data, $message, $code);
    }

    /**
     * 操作失败返回数据
     * @param string $message 错误信息
     * @param int $code 状态码
     * @param mixed $data 返回数据
     * @return \think\Response
     */
    protected function error($message = 'error', $code = 400, $data = null)
    {
        return Result::error($message, $code, $data);
    }

    /**
     * 获取请求参数
     * @param array $rules 验证规则
     * @param array $message 验证消息
     * @return array
     */
    protected function getRequestData($rules = [], $message = [])
    {
        // 调试：记录原始请求
        $url = request()->url();
        Log::info("获取请求数据参数 - URL: " . $url);

        $data = request()->post();

        if (empty($data)) {
            $raw = file_get_contents('php://input');
            Log::info("从php://input读取原始数据: " . $raw);
            $json = json_decode($raw, true);
            if (is_array($json)) {
                $data = $json;
            }
        }

        Log::info("获取请求数据 - 解析后的数据: " . json_encode($data, JSON_UNESCAPED_UNICODE));

        if (!empty($rules)) {
            $validate = validate()->rule($rules)->message($message);
            if (!$validate->check($data)) {
                throw new \Exception($validate->getError());
            }
        }

        return $data;
    }
}
```

```
protected function getUserId()
{
    $userId = request()->userId ?? null;
    if (!$userId) {
        throw new \Exception('未登录');
    }
    return intval($userId);
}
}
```

// backend/app/controller/BindingController.php

```
<?php
```

```
namespace app\controller;
```

```
use app\model\Personnel;
use app\model\Department;
use app\model\User;
use app\model\Engineer;
use app\model\Role;
use app\model\RolePermission;
use app\model\UserRole;
use app\common\Result;
```

```
/**
```

```
 * 绑定解绑管理控制器
```

```
 */
```

```
class BindingController
```

```
{
```

```
    /**
```

```
     * 获取人员部门绑定列表
```

```
     * GET /bindings/personnel-department
```

```
     */
```

```
    public function getPersonnelDepartmentList()
```

```
    {
```

```
        $page = request()->get('page', 1);
```

```
        $pageSize = request()->get('pageSize', 20);
```

```
        $personnelName = request()->get('personnel_name', '');
```

```
        $departmentName = request()->get('department_name', '');
```

```
        try {
```

```
            $query = Personnel::alias('p')
```

```
                ->field('p.id as personnel_id, p.name as personnel_name, p.code as personnel_code, p.position as personnel_position, p.department_id, d.name as department_name, p.updated_at as bound_at')
```

```
                ->leftJoin('departments d', 'p.department_id = d.id')
```

```
                ->order('p.id', 'desc');
```

```
            if (!empty($personnelName)) {
```

```
                $query->whereLike('p.name', '%' . $personnelName . '%');
```

```
            }
```

```
            if (!empty($departmentName)) {
```

```
                $query->whereLike('d.name', '%' . $departmentName . '%');
```

```
            }
```

```
            $total = $query->count();
```

```
            $list = $query->page($page, $pageSize)->select();
```

```
            return Result::paginated($list, $total, $page, $pageSize);
```

```
        } catch (\Exception $e) {
```

```
        return Result::error($e->getMessage(), 500);
    }
}

/**
 * 绑定人员到部门
 * POST /bindings/personnel-department
 */
public function bindPersonnelToDepartment()
{
    $data = request()->post();

    try {
        validate([
            'personnel_id' => 'require|integer',
            'department_id' => 'require|integer',
        ]->check($data);

        $personnel = Personnel::find($data['personnel_id']);
        if (!$personnel) {
            return Result::error('人员不存在', 404);
        }

        $department = Department::find($data['department_id']);
        if (!$department) {
            return Result::error('部门不存在', 404);
        }

        $personnel->department_id = $data['department_id'];
        $personnel->save();

        return Result::success($personnel, '绑定成功');
    } catch (\think\Exception\ValidateException $e) {
        return Result::error($e->getError(), 422);
    } catch (\Exception $e) {
        return Result::error($e->getMessage(), 500);
    }
}

/**
 * 解绑人员与部门
 * DELETE /bindings/personnel-department/{personnelId}
 */
public function unbindPersonnelFromDepartment($personnelId)
{
    try {
        $personnel = Personnel::find($personnelId);
        if (!$personnel) {
            return Result::error('人员不存在', 404);
        }

        $personnel->department_id = null;
        $personnel->save();

        return Result::success(null, '解绑成功');
    } catch (\Exception $e) {
        return Result::error($e->getMessage(), 500);
    }
}

/**
 * 获取工程师用户绑定列表
 * GET /bindings/engineer-user
```

```

*/
public function getEngineerUserList()
{
    $page = request()->get('page', 1);
    $pageSize = request()->get('pageSize', 20);
    $username = request()->get('username', '');
    $bound = request()->get('bound', '');

    try {
        $query = User::alias('u')
            ->field('u.id as user_id, u.username, u.real_name, u.phone, e.id as engineer_id, e.skill_level, e.specialties,
e.work_years, e.certification, e.status, e.updated_at as bound_at')
            ->leftJoin('engineers e', 'u.id = e.user_id')
            ->order('u.id', 'desc');

        if (!empty($username)) {
            $query->where('u.username|u.real_name', 'like', '%' . $username . '%');
        }

        if ($bound === 'true') {
            $query->where('e.id', '<>', null);
        } elseif ($bound === 'false') {
            $query->where('e.id', '=', null);
        }

        $total = $query->count();
        $list = $query->page($page, $pageSize)->select();

        return Result::paginated($list, $total, $page, $pageSize);
    } catch (\Exception $e) {
        return Result::error($e->getMessage(), 500);
    }
}

/**
 * 绑定工程师与用户
 * POST /bindings/engineer-user
 */
public function bindEngineerToUser()
{
    $data = request()->post();

    try {
        validate([
            'user_id' => 'require|integer',
            'skill_level' => 'require|integer|between:1,4',
        ])->check($data);

        $user = User::find($data['user_id']);
        if (!$user) {
            return Result::error('用户不存在', 404);
        }

        $engineer = Engineer::where('user_id', $data['user_id'])->find();
        if ($engineer) {
            $engineer->skill_level = $data['skill_level'];
            $engineer->specialties = $data['specialties'] ?? [];
            $engineer->work_years = $data['work_years'] ?? 0;
            $engineer->certification = $data['certification'] ?? '';
            $engineer->status = $data['status'] ?? 1;
            $engineer->save();
        } else {
            $engineer = Engineer::create([

```

```

        'user_id' => $data['user_id'],
        'skill_level' => $data['skill_level'],
        'specialties' => $data['specialties'] ?? [],
        'work_years' => $data['work_years'] ?? 0,
        'certification' => $data['certification'] ?? '',
        'status' => $data['status'] ?? 1,
    ));
}

return Result::success($engineer, '绑定成功');
} catch (\think\Exception\ValidateException $e) {
    return Result::error($e->getError(), 422);
} catch (\Exception $e) {
    return Result::error($e->getMessage(), 500);
}
}

/**
 * 解绑工程师与用户
 * DELETE /bindings/engineer-user/{engineerId}
 */
public function unbindEngineerFromUser($engineerId)
{
    try {
        $engineer = Engineer::find($engineerId);
        if (!$engineer) {
            return Result::error('工程师不存在', 404);
        }

        $engineer->delete();

        return Result::success(null, '解绑成功');
    } catch (\Exception $e) {
        return Result::error($e->getMessage(), 500);
    }
}

/**
 * 获取用户角色绑定列表
 * GET /bindings/user-role
 */
public function getUserRoleList()
{
    $page = request()->get('page', 1);
    $pageSize = request()->get('pageSize', 20);
    $username = request()->get('username', '');
    $roleName = request()->get('role_name', '');
    $status = request()->get('status', '');

    try {
        $query = User::alias('u')
            ->field('u.id as user_id, u.username, u.real_name, u.phone, u.status, u.last_login_at as last_login, d.name
as department_name')
            ->leftJoin('departments d', 'u.department_id = d.id')
            ->order('u.id', 'desc');

        if (!empty($username)) {
            $query->where('u.username|u.real_name', 'like', '%' . $username . '%');
        }

        if (!empty($status)) {
            $query->where('u.status', $status);
        }
    }
}

```

```

// 如果指定了角色名称, 需要先获取具有该角色的用户ID
if (!empty($roleName)) {
    $roleUserIds = UserRole::alias('ur')
        ->leftJoin('roles r', 'ur.role_id = r.id')
        ->whereLike('r.name', '%' . $roleName . '%')
        ->column('ur.user_id');

    if (empty($roleUserIds)) {
        // 如果没有用户匹配该角色, 直接返回空结果
        return Result::paginated([], 0, $page, $pageSize);
    }
    $query->whereIn('u.id', $roleUserIds);
}

$total = $query->count();
$users = $query->page($page, $pageSize)->select();

// 获取每个用户的角色
foreach ($users as &$user) {
    $userRoles = UserRole::alias('ur')
        ->leftJoin('roles r', 'ur.role_id = r.id')
        ->where('ur.user_id', $user['user_id'])
        ->field('r.id, r.name, r.status')
        ->select();
    $user['roles'] = $userRoles ?: [];
}

return Result::paginated($users, $total, $page, $pageSize);
} catch (\Exception $e) {
    return Result::error($e->getMessage(), 500);
}
}

/**
 * 绑定用户到角色
 * POST /bindings/user-role
 */
public function bindUserToRole()
{
    $data = request()->post();

    try {
        validate([
            'user_id' => 'require|integer',
            'role_id' => 'require|integer',
        ])->check($data);

        $user = User::find($data['user_id']);
        if (!$user) {
            return Result::error('用户不存在', 404);
        }

        $role = Role::find($data['role_id']);
        if (!$role) {
            return Result::error('角色不存在', 404);
        }

        // 如果是替换模式, 先删除该用户的所有角色
        if (isset($data['mode']) && $data['mode'] === 'replace') {
            UserRole::where('user_id', $data['user_id'])->delete();
        }
    }
}

```

```

// 检查是否已存在该绑定
$existing = UserRole::where('user_id', $data['user_id'])
    ->where('role_id', $data['role_id'])
    ->find();

if (!$existing) {
    UserRole::create([
        'user_id' => $data['user_id'],
        'role_id' => $data['role_id'],
    ]);
}

return Result::success(null, '绑定成功');
} catch (\think\exception\ValidateException $e) {
    return Result::error($e->getError(), 422);
} catch (\Exception $e) {
    return Result::error($e->getMessage(), 500);
}
}

/**
 * 解绑用户与角色
 * DELETE /bindings/user-role/{userId}/{roleId}
 */
public function unbindUserFromRole($userId, $roleId)
{
    try {
        $binding = UserRole::where('user_id', $userId)
            ->where('role_id', $roleId)
            ->find();

        if (!$binding) {
            return Result::error('绑定关系不存在', 404);
        }

        $binding->delete();

        return Result::success(null, '解绑成功');
    } catch (\Exception $e) {
        return Result::error($e->getMessage(), 500);
    }
}
}

```

// agent-service/app.py

```

import time
import uuid
from typing import Any, Dict, List

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field

from graph_agent import invoke_agent, log_debug

MAX_MESSAGE_LENGTH = 4000
MAX_HISTORY_MESSAGES = 12
MAX_HISTORY_CONTENT_LENGTH = 2000
ALLOWED_HISTORY_ROLES = {"user", "assistant"}

```

```
class ChatRequest(BaseModel):
    message: str
    history: List[Dict[str, str]] = Field(default_factory=list)
    user_id: int | None = None

app = FastAPI(title="CMMS Agent Service")

def normalize_message(value: str) -> str:
    message = (value or "").strip()
    if not message:
        raise ValueError("消息内容不能为空")
    if len(message) > MAX_MESSAGE_LENGTH:
        raise ValueError(f"消息内容不能超过 {MAX_MESSAGE_LENGTH} 个字符")
    return message

def normalize_history(history: List[Dict[str, str]]) -> List[Dict[str, str]]:
    normalized: List[Dict[str, str]] = []
    for item in history[-MAX_HISTORY_MESSAGES:]:
        if not isinstance(item, dict):
            continue
        role = str(item.get("role", "")).strip()
        content = str(item.get("content", "")).strip()
        if role not in ALLOWED_HISTORY_ROLES or not content:
            continue
        normalized.append(
            {
                "role": role,
                "content": content[:MAX_HISTORY_CONTENT_LENGTH],
            }
        )
    return normalized

@app.get("/health")
def health() -> Dict[str, str]:
    return {"status": "ok"}

@app.post("/chat")
def chat(request: ChatRequest) -> Dict[str, Any]:
    request_id = uuid.uuid4().hex[:12]
    started_at = time.perf_counter()
    try:
        message = normalize_message(request.message)
        history = normalize_history(request.history)
        log_debug(
            "chat-request",
            {
                "request_id": request_id,
                "message": message,
                "history_count": len(history),
                "user_id": request.user_id,
            },
        )
        response = invoke_agent(
            message=message,
            history=history,
            user_id=request.user_id,
        )
```

```

    log_debug(
        "chat-response",
        {
            "request_id": request_id,
            "message": message,
            "tools_used": response.get("tools_used", []),
            "scenario": response.get("scenario"),
            "duration_ms": int((time.perf_counter() - started_at) * 1000),
        },
    )
    return {
        **response,
        "request_id": request_id,
    }
except ValueError as exc:
    raise HTTPException(status_code=422, detail=str(exc)) from exc
except Exception as exc:
    raise HTTPException(status_code=500, detail=str(exc)) from exc

```

```

if __name__ == "__main__":
    import os

    import uvicorn

    uvicorn.run(app, host="0.0.0.0", port=int(os.getenv("AGENT_SERVICE_PORT", "8001")))

```

// agent-service/data_access.py

```

import os
import socket
from typing import Any, Dict, List

from dotenv import load_dotenv
from sqlalchemy import create_engine, text

BASE_DIR = os.path.dirname(__file__)

# 优先使用系统环境变量；本地开发时再从 .env 文件补齐缺失配置。
load_dotenv(os.path.join(BASE_DIR, ".env"), override=False)
load_dotenv(os.path.join(BASE_DIR, "..", "backend", ".env"), override=False)

def env(name: str, default: str = "") -> str:
    return os.getenv(name, default)

def can_resolve(host: str) -> bool:
    try:
        socket.gethostbyname(host)
        return True
    except OSError:
        return False

def resolve_db_host(host: str) -> str:
    if host in {"mysql", "mariadb"} and not can_resolve(host):
        return env("AGENT_DB_HOST_FALLBACK", "127.0.0.1")
    return host

```

```
def mysql_url(prefix: str) -> str:
    host = resolve_db_host(env(f"{prefix}_HOSTNAME", "127.0.0.1"))
    port = env(f"{prefix}_HOSTPORT", "3306")
    database = env(f"{prefix}_DATABASE", "")
    username = env(f"{prefix}_USERNAME", "root")
    password = env(f"{prefix}_PASSWORD", "")
    return f"mysql+pymysql://{username}:{password}@{host}:{port}/{database}?charset=utf8mb4"
```

```
CMMS_ENGINE = create_engine(mysql_url("DATABASE"), pool_pre_ping=True)
REPAIR_ENGINE = create_engine(mysql_url("REPAIR_DB"), pool_pre_ping=True)
```

```
def fetch_rows(engine, sql: str, params: Dict[str, Any] | None = None) -> List[Dict[str, Any]]:
    with engine.connect() as conn:
        result = conn.execute(text(sql), params or {})
        return [dict(row._mapping) for row in result]
```

```
def fetch_one(engine, sql: str, params: Dict[str, Any] | None = None) -> Dict[str, Any] | None:
    rows = fetch_rows(engine, sql, params)
    return rows[0] if rows else None
```

```
def safe_fetch_rows(engine, sql: str, params: Dict[str, Any] | None = None) -> List[Dict[str, Any]]:
    try:
        return fetch_rows(engine, sql, params)
    except Exception:
        return []
```

```
def fetch_rows_with_fallback(
    primary_engine,
    sql: str,
    params: Dict[str, Any] | None = None,
    *,
    fallback_engine=None,
) -> List[Dict[str, Any]]:
    try:
        return fetch_rows(primary_engine, sql, params)
    except Exception:
        if fallback_engine is None:
            raise
        return fetch_rows(fallback_engine, sql, params)
```

```
def query_user_context(user_id: int) -> Dict[str, Any]:
    repair_user = safe_fetch_rows(
        REPAIR_ENGINE,
        """
        SELECT id, username, nickname, real_name, phone, email, role, status
        FROM users
        WHERE id = :user_id
        LIMIT 1
        """,
        {"user_id": user_id},
    )
    recent_repair_orders = safe_fetch_rows(
        REPAIR_ENGINE,
        """
        SELECT id, order_id AS order_no, status, updated_at, device_model
        FROM orders
        WHERE assigned_to = :user_id OR quote_created_by = :user_id
        """
```

```

ORDER BY updated_at DESC, id DESC
LIMIT 5
"""
{"user_id": user_id},
)
recent_cmms_orders = safe_fetch_rows(
    CMMS_ENGINE,
    """
SELECT id, order_no, status, updated_at, fault_type
FROM work_orders
WHERE assigned_to = :user_id OR reporter_id = :user_id
ORDER BY updated_at DESC, id DESC
LIMIT 5
"""
{"user_id": user_id},
)
return {
    "repair_user": repair_user[0] if repair_user else None,
    "recent_repair_orders": recent_repair_orders,
    "recent_cmms_work_orders": recent_cmms_orders,
}

```

```

def query_personnel(keyword: str, limit: int = 10) -> List[Dict[str, Any]]:
    return fetch_rows_with_fallback(
        REPAIR_ENGINE,
        """
SELECT
    u.id,
    COALESCE(u.real_name, u.nickname, CONCAT('用户#', u.id)) AS name,
    u.nickname,
    u.real_name,
    u.phone,
    u.email,
    u.status,
    u.role,
    NULL AS department_name
FROM users u
WHERE
    COALESCE(u.real_name, '') LIKE :kw
    OR COALESCE(u.nickname, '') LIKE :kw
    OR COALESCE(u.phone, '') LIKE :kw
    OR COALESCE(u.email, '') LIKE :kw
    OR COALESCE(u.role, '') LIKE :kw
ORDER BY u.role DESC, u.status DESC, u.id DESC
LIMIT :limit
"""
{"kw": f"%{keyword}%", "limit": limit},
        fallback_engine=CMMS_ENGINE,
    )

```

```

def query_repair_personnel(keyword: str = "", limit: int = 10) -> List[Dict[str, Any]]:
    return fetch_rows_with_fallback(
        REPAIR_ENGINE,
        """
SELECT
    u.id,
    COALESCE(u.real_name, u.nickname, CONCAT('用户#', u.id)) AS name,
    u.nickname,
    u.real_name,
    u.phone,
    u.email,

```

```

        u.status,
        u.role,
        NULL AS department_name
    FROM users u
    WHERE
        u.role IN ('admin', 'super_admin')
        AND (
            :kw = ""
            OR COALESCE(u.real_name, "") LIKE :like_kw
            OR COALESCE(u.nickname, "") LIKE :like_kw
            OR COALESCE(u.phone, "") LIKE :like_kw
            OR COALESCE(u.email, "") LIKE :like_kw
            OR COALESCE(u.role, "") LIKE :like_kw
        )
    ORDER BY
        CASE WHEN u.role = 'super_admin' THEN 0 ELSE 1 END,
        u.status DESC,
        u.id DESC
    LIMIT :limit
    """
    {"kw": keyword, "like_kw": f"%{keyword}%", "limit": limit},
    fallback_engine=CMMS_ENGINE,
)

```

```

def query_orders(keyword: str, limit: int = 10) -> List[Dict[str, Any]]:
    return fetch_rows(
        REPAIR_ENGINE,
        """
        SELECT
            id,
            order_id AS order_no,
            NULL AS customer_name,
            device_model AS machine_name,
            order_type AS repair_type,
            status AS repair_status,
            created_at,
            updated_at
        FROM orders
        WHERE order_id LIKE :kw
            OR device_model LIKE :kw
            OR problem_description LIKE :kw
            OR status LIKE :kw
        ORDER BY updated_at DESC, id DESC
        LIMIT :limit
        """
        {"kw": f"%{keyword}%", "limit": limit},
    )

```

```

def query_repair_order_detail(order_id: int) -> List[Dict[str, Any]]:
    return fetch_rows(
        REPAIR_ENGINE,
        """
        SELECT
            o.id,
            o.order_id AS order_no,
            o.status,
            o.updated_at,
            o.created_at,
            o.progress,
            o.progress_updated_at,
            o.device_model,

```

```

        o.problem_description,
        o.assigned_to,
        COALESCE(u.real_name, u.nickname, CONCAT('用户#', o.assigned_to)) AS assigned_to_name
    FROM orders o
    LEFT JOIN users u ON u.id = o.assigned_to
    WHERE o.id = :order_id
    LIMIT 1
    """
    {"order_id": order_id},
)

```

```

def query_repair_order_workload(message: str, limit: int = 5) -> List[Dict[str, Any]]:
    wants_completed = any(token in message for token in ["完成", "已完成", "完工", "修好"])
    wants_quoted = "报价" in message or "quote" in message.lower()
    wants_pending = any(token in message for token in ["待处理", "待接单", "未完成", "处理中"])
    wants_top = not any(token in message for token in ["最少", "倒数"])
    aggregate_field = "quote_created_by" if wants_quoted else "assigned_to"

    extra_where = " AND o.order_type = 'repair' "
    if wants_completed:
        extra_where += " AND o.status = 'completed' "
    elif wants_quoted:
        extra_where += " AND o.quote_created_by IS NOT NULL "
    elif wants_pending:
        extra_where += " AND o.status IN ('pending', 'quoted', 'confirmed', 'processing') "

    return fetch_rows(
        REPAIR_ENGINE,
        """
        SELECT
            o.{aggregate_field} AS technician_id,
            COALESCE(u.real_name, u.nickname, CONCAT('用户#', o.{aggregate_field})) AS technician_name,
            COUNT(*) AS order_count,
            SUM(CASE WHEN o.status = 'completed' THEN 1 ELSE 0 END) AS completed_count,
            SUM(CASE WHEN o.status = 'processing' THEN 1 ELSE 0 END) AS processing_count,
            SUM(CASE WHEN o.status = 'quoted' THEN 1 ELSE 0 END) AS quoted_count
        FROM orders o
        LEFT JOIN users u ON u.id = o.{aggregate_field}
        WHERE o.{aggregate_field} IS NOT NULL
            AND o.{aggregate_field} <> 0
            {extra_where}
        GROUP BY o.{aggregate_field}, technician_name
        ORDER BY order_count {"DESC" if wants_top else "ASC"}, technician_id ASC
        LIMIT :limit
        """
        {"limit": limit},
    )

```

```

def query_work_orders(keyword: str = "", limit: int = 10) -> List[Dict[str, Any]]:
    return fetch_rows(
        CMMS_ENGINE,
        """
        SELECT
            wo.id,
            wo.order_no,
            wo.fault_type,
            wo.fault_description,
            wo.priority,
            wo.status,
            wo.total_cost,
            wo.created_at,

```

```

        wo.updated_at,
        reporter.name AS reporter_name,
        assignee.name AS assigned_to_name
    FROM work_orders wo
    LEFT JOIN personnel reporter ON reporter.id = wo.reporter_id
    LEFT JOIN personnel assignee ON assignee.id = wo.assigned_to
    WHERE (:kw = '' OR wo.order_no LIKE :like_kw OR wo.fault_type LIKE :like_kw OR wo.fault_description LIKE :like_kw)
    ORDER BY wo.updated_at DESC, wo.id DESC
    LIMIT :limit
    """
    {"kw": keyword, "like_kw": f"%{keyword}%", "limit": limit},
)

```

```
def query_progress(order_id: int) -> List[Dict[str, Any]]:
```

```

    return fetch_rows_with_fallback(
        REPAIR_ENGINE,
        """
        SELECT
            id,
            order_id,
            stage AS progress_stage,
            stage_name AS progress_title,
            status,
            progress AS progress_percent,
            description,
            handler_name AS assigned_to_name,
            start_time AS started_at,
            end_time AS completed_at,
            updated_at
        FROM repair_progress
        WHERE order_id = :order_id
        ORDER BY id ASC
        """
        ,
        {"order_id": order_id},
        fallback_engine=CMMS_ENGINE,
    )

```

```
def query_inventory(keyword: str = "", low_stock_only: bool = False, limit: int = 10) -> List[Dict[str, Any]]:
```

```

    conditions = []
    params: Dict[str, Any] = {"limit": limit}
    if keyword:
        conditions.append("(sp.part_name LIKE :kw OR sp.part_code LIKE :kw OR sp.specification LIKE :kw OR sp.description LIKE :kw)")
        params["kw"] = f"%{keyword}%"
    if low_stock_only:
        conditions.append("sp.stock_quantity <= sp.min_stock")

    where_sql = f"WHERE {' AND ' .join(conditions)}" if conditions else ""
    return fetch_rows_with_fallback(
        REPAIR_ENGINE,
        f"""
        SELECT
            sp.id,
            sp.part_code,
            sp.part_name,
            sp.specification,
            sp.stock_quantity,
            sp.min_stock,
            sp.purchase_price,
            s.name AS supplier_name
        """
    )

```

```

FROM spare_parts sp
LEFT JOIN suppliers s ON s.id = sp.supplier_id
{where_sql}
ORDER BY (sp.stock_quantity <= sp.min_stock) DESC, sp.stock_quantity ASC, sp.id DESC
LIMIT :limit
"""
,
params,
fallback_engine=CMMS_ENGINE,
)

```

```

def query_suppliers(keyword: str = "", limit: int = 10) -> List[Dict[str, Any]]:
    return fetch_rows_with_fallback(
        REPAIR_ENGINE,
        """
        SELECT id, name, code, contact_person, contact_phone AS phone, contact_email AS email, status
        FROM suppliers
        WHERE (:kw = "" OR name LIKE :like_kw OR code LIKE :like_kw)
        ORDER BY status DESC, id DESC
        LIMIT :limit
        """,
        {"kw": keyword, "like_kw": f"%{keyword}%", "limit": limit},
        fallback_engine=CMMS_ENGINE,
    )

```

```

def query_supplier_inventory_ranking(limit: int = 10) -> List[Dict[str, Any]]:
    return fetch_rows_with_fallback(
        REPAIR_ENGINE,
        """
        SELECT
            s.id,
            s.name,
            s.code,
            COUNT(sp.id) AS part_count,
            COALESCE(SUM(sp.stock_quantity * sp.purchase_price), 0) AS inventory_value,
            COALESCE(SUM(sp.stock_quantity), 0) AS total_stock_quantity
        FROM suppliers s
        LEFT JOIN spare_parts sp ON sp.supplier_id = s.id
        GROUP BY s.id, s.name, s.code
        HAVING COUNT(sp.id) > 0
        ORDER BY inventory_value DESC, part_count DESC, s.id DESC
        LIMIT :limit
        """,
        {"limit": limit},
        fallback_engine=CMMS_ENGINE,
    )

```

```

def query_business_overview(keyword: str = "", limit: int = 10) -> Dict[str, Any]:
    return {
        "repair_categories": safe_fetch_rows(
            CMMS_ENGINE,
            """
            SELECT id, name, code, description, status
            FROM repair_categories
            WHERE (:kw = "" OR name LIKE :like_kw OR description LIKE :like_kw)
            ORDER BY sort ASC, id ASC
            LIMIT :limit
            """,
            {"kw": keyword, "like_kw": f"%{keyword}%", "limit": limit},
        ),
        "repair_machines": safe_fetch_rows(

```

```

        CMMS_ENGINE,
        """
        SELECT rm.id, rm.name, rm.model, rm.manufacturer, rm.status, rc.name AS category_name
        FROM repair_machines rm
        LEFT JOIN repair_categories rc ON rc.id = rm.category_id
        WHERE (:kw = "" OR rm.name LIKE :like_kw OR rm.model LIKE :like_kw OR rc.name LIKE :like_kw)
        ORDER BY rm.id DESC
        LIMIT :limit
        """,
        {"kw": keyword, "like_kw": f"%{keyword}%", "limit": limit},
    ),
    "repair_contracts": safe_fetch_rows(
        CMMS_ENGINE,
        """
        SELECT id, contract_number, customer_name, machine_type, annual_fee, status, sign_date, updated_at
        FROM repair_contracts
        WHERE (:kw = "" OR contract_number LIKE :like_kw OR customer_name LIKE :like_kw OR machine_type LIKE
:like_kw OR service_content LIKE :like_kw)
        ORDER BY updated_at DESC, id DESC
        LIMIT :limit
        """,
        {"kw": keyword, "like_kw": f"%{keyword}%", "limit": limit},
    ),
    "repair_reports": safe_fetch_rows(
        CMMS_ENGINE,
        """
        SELECT
            id,
            order_id,
            order_no,
            report_number,
            machine_name,
            fault_description,
            repair_content,
            repairer_name,
            repair_date,
            completion_date,
            amount,
            status,
            remark,
            created_at
        FROM repair_reports
        WHERE (
            :kw = ""
            OR report_number LIKE :like_kw
            OR machine_name LIKE :like_kw
            OR fault_description LIKE :like_kw
            OR repair_content LIKE :like_kw
            OR repairer_name LIKE :like_kw
            OR remark LIKE :like_kw
        )
        ORDER BY id DESC
        LIMIT :limit
        """,
        {"kw": keyword, "like_kw": f"%{keyword}%", "limit": limit},
    ),
}

```

```

def query_knowledge_overview(keyword: str = "", limit: int = 10) -> Dict[str, Any]:
    return {
        "collections": safe_fetch_rows(
            CMMS_ENGINE,

```

```

"""
SELECT id, name, description, file_count, chunk_count, total_chars, status, created_at, updated_at
FROM kb_collections
WHERE (:kw = '' OR name LIKE :like_kw OR description LIKE :like_kw)
ORDER BY id DESC
LIMIT :limit
""",
{"kw": keyword, "like_kw": f"%{keyword}%", "limit": limit},
),
"files": safe_fetch_rows(
    CMMS_ENGINE,
    """
SELECT
    kf.id,
    kf.collection_id,
    kc.name AS collection_name,
    kf.original_name,
    kf.file_type,
    kf.file_size,
    kf.text_char_count,
    kf.chunk_count,
    kf.chunk_status,
    kf.created_at
FROM kb_files kf
LEFT JOIN kb_collections kc ON kc.id = kf.collection_id
WHERE (
    :kw = ''
    OR kc.name LIKE :like_kw
    OR kf.original_name LIKE :like_kw
    OR kf.extracted_text LIKE :like_kw
)
ORDER BY kf.id DESC
LIMIT :limit
""",
{"kw": keyword, "like_kw": f"%{keyword}%", "limit": limit},
),
"chunks": safe_fetch_rows(
    CMMS_ENGINE,
    """
SELECT
    kc.id,
    kc.collection_id,
    kf.original_name,
    LEFT(kc.content, 220) AS excerpt,
    kc.chunk_index,
    kc.created_at
FROM kb_chunks kc
LEFT JOIN kb_files kf ON kf.id = kc.file_id
WHERE (
    :kw = ''
    OR kc.content LIKE :like_kw
    OR kf.original_name LIKE :like_kw
)
ORDER BY kc.id DESC
LIMIT :limit
""",
{"kw": keyword, "like_kw": f"%{keyword}%", "limit": limit},
),
}

```

```

def query_rag(question: str, limit: int = 5) -> List[Dict[str, Any]]:
    tokens = [token for token in question.split() if len(token) >= 2]

```

```

if not tokens:
    tokens = [question[:20]] if question.strip() else ["知识库"]

where_parts = []
params: Dict[str, Any] = {"limit": limit}
for index, token in enumerate(tokens[:4]):
    key = f"kw{index}"
    where_parts.append(f"(kc.content LIKE :{key} OR kf.original_name LIKE :{key})")
    params[key] = f"%{token}%"

where_sql = " OR ".join(where_parts) if where_parts else "1=1"
return fetch_rows(
    CMMS_ENGINE,
    f"""
    SELECT kc.content, kf.original_name
    FROM kb_chunks kc
    LEFT JOIN kb_files kf ON kf.id = kc.file_id
    WHERE {where_sql}
    ORDER BY kc.id DESC
    LIMIT :limit
    """,
    params,
)

```

// agent-service/graph_agent.py

```

import json
import os
import re
from datetime import datetime
from functools import lru_cache
from typing import Any, Dict, List, Literal, TypedDict

from langchain_core.messages import AIMessage, BaseMessage, HumanMessage, SystemMessage
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI
from langgraph.graph import END, START, StateGraph

from data_access import (
    env,
    query_business_overview,
    query_inventory,
    query_knowledge_overview,
    query_orders,
    query_personnel,
    query_progress,
    query_rag,
    query_repair_order_detail,
    query_repair_order_workload,
    query_repair_personnel,
    query_supplier_inventory_ranking,
    query_suppliers,
    query_user_context,
    query_work_orders,
)

```

```

SYSTEM_FEATURES = {
    "基础管理": ["用户管理", "角色管理", "权限管理", "人员管理", "单位管理", "日志管理", "参数管理"],
    "业务管理": ["免责协议管理", "常见问题管理", "绑定/解绑"],
    "引流模块": ["成功案例", "人工客服", "抖音获客", "小红书获客", "快手获客", "B站获客", "合作企业"],

```

```

"维修业务": ["机械种类管理", "机械名称管理", "订单管理", "检测报告", "维修报告", "合同管理", "维修提醒", "联动维修",
"维修进度"],
"支付模块": ["转账支付", "在线支付", "支付宝测试", "发票管理"],
"进销存": ["配件管理", "供应商管理"],
"查询统计": ["收入统计", "开支统计", "订单统计", "超时统计"],
"知识库": ["知识库管理", "AI 对话"],
"Agent": ["系统智能体"],
}

```

```

MODULE_ALIASES = {
"基础管理": ["基础管理", "基础模块", "用户管理", "角色管理", "权限管理", "人员管理", "单位管理", "日志管理", "参数管
理"],
"业务管理": ["业务管理", "业务模块", "免责协议", "常见问题", "绑定", "解绑"],
"引流模块": ["引流模块", "引流", "营销模块", "成功案例", "人工客服", "抖音获客", "小红书获客", "快手获客", "b站获客",
"合作企业"],
"维修业务": ["维修业务", "维修模块", "维修系统", "维修功能", "机械种类", "机械名称", "订单管理", "检测报告", "维修报
告", "合同管理", "维修提醒", "联动维修", "维修进度"],
"支付模块": ["支付模块", "支付", "转账支付", "在线支付", "支付宝测试", "发票管理"],
"进销存": ["进销存", "库存模块", "库存管理", "配件管理", "供应商管理", "配件", "备件", "库存", "供应商"],
"查询统计": ["查询统计", "统计模块", "统计", "收入统计", "开支统计", "订单统计", "超时统计"],
"知识库": ["知识库", "知识库模块", "知识内容", "知识文件", "知识库管理", "ai 对话", "文档"],
"Agent": ["agent", "智能体", "系统智能体"],
}

```

```

STOP_WORDS = [
"查询", "查一下", "查一查", "帮我查", "帮忙查", "请问", "请", "一下", "现在", "当前", "目前",
"所有", "全部", "有哪些", "有什么", "信息", "情况", "数据", "记录", "列表", "内容", "状态",
"分析", "统计", "概括", "看看", "显示", "列出", "帮我", "关于", "相关", "里面", "一下子", "的",
"请基于", "基于", "列出", "说明", "总结", "排序", "优先", "并按", "并给出", "当前到哪个阶段",
"最近一次更新", "负责人是谁", "前", "名", "系统里", "页面入口", "关键流程节点", "小程序",
"repair", "cmms_db", "数据库",
]

```

```

SUGGESTION_NORMALIZERS = [
(["低库存", "库存"], "请优先说明当前库存、最低库存和补货优先级。"),
(["订单", "进度"], "请先回答当前状态，再给最近更新时间、负责人和下一步阶段。"),
(["供应商", "库存金额"], "请按库存金额排序，并补充配件种类数。"),
(["知识库", "维修流程"], "请先概括主流程，再列关键节点和注意事项。"),
(["系统", "模块"], "请按模块说明入口、能力和常见用途。"),
]

```

```

SCENARIO_PROMPTS = {
"repair_order": "这是企业维修订单场景。先给结论，再说明订单状态、负责人、进度、更新时间、异常点和下一步建议",
,
"work_order": "这是企业 CMMS 工单场景。优先给出工单状态、优先级、故障描述、责任人和处理建议。",
"inventory": "这是企业库存或供应商场景。优先给出结论、查询范围、关键数据、风险判断以及补货或供应建议。",
"personnel": "这是企业人员场景。优先给出人员身份、角色、联系方式、工作负载和适合的管理动作。",
"knowledge": "这是企业知识库或流程说明场景。优先基于知识库给出摘要、适用模块、关键流程节点和落地建议。",
"overview": "这是企业系统模块概览场景。优先按侧边栏模块、页面入口、核心能力、关联数据和使用场景来组织回答。",
,
"general": "这是企业综合问答场景。回答要像后台管理智能助手，先结论，再依据，再建议。",
}

```

ENTERPRISE_SYSTEM_PROMPT = ""你是企业级维修管理系统的系统智能体，服务对象是管理员、运营人员、仓库人员和维修负责人。

回答要求：

1. 回答风格必须专业、克制、可执行，像企业后台智能助手，不要像泛用聊天机器人。
2. 始终优先依据系统真实数据回答，禁止编造业务数据、人员、订单、库存或流程结果。
3. 默认回答结构：
 - 先给结论
 - 再给关键依据
 - 最后给下一步建议或可继续追问方向

4. 如果结果为空，不要只说“未查到”。

需要明确：

- 本次查询查了什么范围
- 当前为什么没有命中结果
- 用户下一步可以怎么查

5. 如果结果不完整，要明确说“当前仅查到部分数据”或“当前结果主要来自某个数据库”。

6. repair 数据库侧重维修订单、维修进度、库存、供应商、维修用户；cmms_db 侧重系统模块、知识库、CMMS 工单、维修业务配置。

7. 输出使用简洁中文，但信息密度要高，适合企业管理场景。""

```
STATUS_LABELS = {
    "pending": "待处理",
    "quoted": "待确认报价",
    "confirmed": "已确认报价",
    "processing": "处理中",
    "completed": "已完成",
    "cancelled": "已取消",
}

MAX_AGENT_HISTORY = 12
MAX_HISTORY_ITEM_LENGTH = 2000
```

```
class AgentState(TypedDict, total=False):
    user_id: int | None
    user_profile: Dict[str, Any]
    history: List[Dict[str, str]]
    message: str
    normalized_message: str
    analysis: Dict[str, Any]
    tool_outputs: List[Dict[str, Any]]
    tools_used: List[str]
    tool_errors: List[Dict[str, Any]]
    answer: str
```

```
def is_sidebar_module_question(message: str) -> bool:
    module_tokens = ["侧边栏", "菜单", "模块", "页面", "页面入口", "系统里", "系统中", "功能入口"]
    return any(token in message for token in module_tokens)
```

```
def is_miniprogram_repair_question(message: str) -> bool:
    miniprogram_tokens = ["小程序", "客户订单", "维修订单", "维修人员", "维修工程师", "维修负责人", "报价"]
    return any(token in message for token in miniprogram_tokens)
```

```
def extract_order_search_keyword(message: str) -> str:
    explicit_no = re.search(r"(?:订单号|工单号)\s*[:\s*#]?([A-Za-z0-9-_.]+)", message)
    if explicit_no:
        return explicit_no.group(1)
    numeric_no = re.search(r"(?:订单|工单)\s*([0-9]{1,10})", message)
    if numeric_no:
        return numeric_no.group(1)
    keyword = extract_named_keyword(
        message,
        [
            "订单", "工单", "维修单", "小程序", "维修订单", "客户订单", "当前", "最近", "状态", "进度",
            "负责人", "维修人员", "最近一次更新", "当前到哪个阶段", "说明", "列出", "所有", "全部",
        ],
    )
    return keyword
```

```

def is_general_order_lookup(message: str) -> bool:
    broad_patterns = [
        "订单有哪些",
        "有哪些订单",
        "订单信息",
        "查看订单",
        "订单列表",
        "订单情况",
        "订单状态",
        "订单量",
    ]
    return any(token in message for token in broad_patterns)

def log_debug(event: str, payload: Dict[str, Any]) -> None:
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    try:
        print(f"[agent-service][{timestamp}][{event}] {json.dumps(payload, ensure_ascii=False, default=str)}")
    except Exception:
        print(f"[agent-service][{timestamp}][{event}] {payload}")

def cleanup_keyword(text_value: str) -> str:
    value = text_value.strip()
    for token in STOP_WORDS:
        value = value.replace(token, " ")
    value = re.sub(r"[ , . ! ? ; : , ! ? ; / \ ( ) 【 】 \\\\" + " " + " " + " ]+", " ", value)
    value = re.sub(r"\\s+", " ", value).strip()
    return value if len(value) > 1 else ""

def normalize_message(message: str) -> str:
    normalized = message.strip()
    compact = cleanup_keyword(normalized)
    for tokens, enhancement in SUGGESTION_NORMALIZERS:
        if all(token in normalized for token in tokens) or all(token in compact for token in tokens):
            return f"{normalized}\\n\\n补充要求: {enhancement}"
    return normalized

def detect_modules(message: str) -> List[str]:
    normalized = cleanup_keyword(message).lower() or message.strip().lower()
    matched: List[str] = []
    for module_name, aliases in MODULE_ALIASES.items():
        alias_pool = [module_name.lower(), *[alias.lower() for alias in aliases]]
        if any(alias and alias in normalized for alias in alias_pool):
            if module_name == "业务管理" and any(token in normalized for token in ["维修业务", "维修模块", "维修业务模块"]):
                continue
            matched.append(module_name)
    return matched

def detect_order_id(message: str) -> int | None:
    match = re.search(r"(?:订单|工单)\\s*([0-9]{1,10})", message)
    if match:
        return int(match.group(1))
    return None

def extract_status_keyword(message: str) -> str:
    status_map = {
        "待处理": "pending",
    }

```

```

    "处理中": "processing",
    "待维修": "pending",
    "已完成": "completed",
    "完成": "completed",
    "已取消": "cancelled",
    "取消": "cancelled",
}
for token, mapped in status_map.items():
    if token in message:
        return mapped
return ""

def extract_named_keyword(message: str, domain_tokens: List[str]) -> str:
    value = cleanup_keyword(message)
    for token in domain_tokens:
        value = value.replace(token, " ")
    return re.sub(r"\s+", " ", value).strip()

def should_use_broad_query(message: str) -> bool:
    broad_tokens = ["所有", "全部", "当前", "现在", "最近", "有哪些", "有什么", "列表", "统计", "情况", "查看订单", "查看维修人员"]
    return any(token in message for token in broad_tokens)

def is_repair_order_analytics_query(message: str) -> bool:
    if not any(token in message for token in ["订单", "工单", "维修单", "接单", "报价"]):
        return False
    patterns = [r"谁.*最多", r"哪个.*最多", r"订单最多", r"接单最多", r"报价最多", r"完成最多", r"排行", r"排名", r"top", r"统计"]
    return any(re.search(pattern, message, re.IGNORECASE) for pattern in patterns)

def is_supplier_inventory_ranking_query(message: str) -> bool:
    if "供应商" not in message and "供货商" not in message:
        return False
    patterns = [r"库存金额最高", r"金额最高", r"排名", r"排行", r"top", r"最高"]
    return any(re.search(pattern, message, re.IGNORECASE) for pattern in patterns)

def is_cmms_work_order_query(message: str) -> bool:
    lower = message.lower()
    cmms_tokens = ["派单", "报修工单", "系统工单", "故障工单", "work order", "fault"]
    repair_tokens = ["维修订单", "用户订单", "报价", "小程序订单", "客户订单"]
    if any(token in lower for token in ["work order", "fault"]):
        return True
    if any(token in message for token in cmms_tokens):
        return True
    if any(token in message for token in repair_tokens):
        return False
    return "工单" in message and "订单" not in message

def is_knowledge_content_question(message: str) -> bool:
    knowledge_tokens = ["知识库", "文档", "文件", "知识内容", "知识库内容", "知识库文件"]
    content_tokens = ["内容", "全文", "文档", "文件", "知识", "说明", "定义", "流程", "规范", "制度", "里面"]
    return any(token in message for token in knowledge_tokens) and any(token in message for token in content_tokens)

def query_system_features(question: str = "", modules: List[str] | None = None) -> Dict[str, List[str]]:
    matched_modules = modules or detect_modules(question)

```

```

    if matched_modules:
        return {module_name: SYSTEM_FEATURES[module_name] for module_name in matched_modules if module_
name in SYSTEM_FEATURES}
    normalized = cleanup_keyword(question)
    if not normalized:
        return SYSTEM_FEATURES
    matched: Dict[str, List[str]] = {}
    for module_name, features in SYSTEM_FEATURES.items():
        if normalized in module_name or any(normalized in feature for feature in features):
            matched[module_name] = features
            continue
        for token in normalized.split():
            if token and len(token) >= 2 and (token in module_name or any(token in feature for feature in features)):
                matched[module_name] = features
                break
    return matched or SYSTEM_FEATURES

```

@tool("analyze_user_message")

def analyze_user_message(payload: str) -> str:

"""

分析用户消息并输出场景、关键词、模块和数据需求。

"""

data = json.loads(payload)

message = data.get("message", "")

normalized = normalize_message(message)

matched_modules = detect_modules(normalized)

sidebar_question = is_sidebar_module_question(normalized)

miniprogram_question = is_miniprogram_repair_question(normalized)

scores = {

"repair_order": 0,

"work_order": 0,

"inventory": 0,

"personnel": 0,

"knowledge": 0,

"overview": 0,

"general": 1,

}

if any(token in normalized for token in ["订单", "维修单", "报价", "进度", "维修负责人"]):

scores["repair_order"] += 4

if miniprogram_question:

scores["repair_order"] += 4

if is_repair_order_analytics_query(normalized):

scores["repair_order"] += 3

if is_cmms_work_order_query(normalized):

scores["work_order"] += 4

if any(token in normalized for token in ["库存", "配件", "备件", "供应商", "低库存"]):

scores["inventory"] += 4

if any(token in normalized for token in ["人员", "员工", "工程师", "负责人", "部门"]):

scores["personnel"] += 3

if miniprogram_question and any(token in normalized for token in ["维修人员", "维修工程师", "维修负责人", "人员"]):

scores["personnel"] += 6

if any(token in normalized for token in ["知识库", "文档", "流程", "规范", "制度"]) or is_knowledge_content_question(normalized):

scores["knowledge"] += 4

if matched_modules or sidebar_question or any(token in normalized for token in ["模块", "菜单", "功能", "入口", "介绍"]):

scores["overview"] += 4

if sidebar_question:

scores["overview"] += 5

if "请基于 cmms_db" in normalized or "cmms_db" in normalized:

scores["overview"] += 3

```

    scores["knowledge"] += 2
    if "请基于 repair" in normalized or "repair 数据库" in normalized:
        scores["repair_order"] += 2
        scores["inventory"] += 2
        scores["personnel"] += 2
    if is_general_order_lookup(normalized):
        scores["repair_order"] += 3

    primary = max(scores, key=scores.get)
    ranked = sorted(scores.items(), key=lambda item: item[1], reverse=True)
    secondary = [name for name, score in ranked if name != primary and score >= 3][:2]
    response = {
        "primary_scenario": primary,
        "secondary_scenarios": secondary,
        "matched_modules": matched_modules,
        "sidebar_question": sidebar_question,
        "miniprogram_question": miniprogram_question,
        "db_preference": (
            "cmms_db"
            if sidebar_question or "cmms_db" in normalized
            else "repair"
            if miniprogram_question or "repair 数据库" in normalized
            else "auto"
        ),
        "order_id": detect_order_id(normalized),
        "status_keyword": extract_status_keyword(normalized),
        "inventory_keyword": extract_named_keyword(normalized, ["低库存", "库存", "配件", "备件", "供应商", "供货商",
"紧急程度", "排序"]),
        "personnel_keyword": extract_named_keyword(normalized, ["人员", "员工", "负责人", "工程师", "部门", "维修人
员", "维修工程师"]),
        "knowledge_keyword": extract_named_keyword(normalized, ["知识库", "文档", "文件内容", "知识内容", "知识库
内容", "知识库文件", "里面", "内容", "查询"]),
        "needs_rag": any(token in normalized for token in ["流程", "规范", "制度", "说明"]) or is_knowledge_content_qu
estion(normalized),
        "needs_progress": "进度" in normalized or "阶段" in normalized,
        "needs_order_detail": detect_order_id(normalized) is not None,
        "needs_workload": is_repair_order_analytics_query(normalized),
        "needs_supplier_ranking": is_supplier_inventory_ranking_query(normalized),
        "use_broad_query": should_use_broad_query(normalized),
    }
    return json.dumps(response, ensure_ascii=False)

```

```

def convert_history(history: List[Dict[str, str]]) -> List[BaseMessage]:
    messages: List[BaseMessage] = []
    for item in sanitize_history(history):
        role = item.get("role")
        content = item.get("content", "")
        if not content:
            continue
        if role == "assistant":
            messages.append(AIMessage(content=content))
        else:
            messages.append(HumanMessage(content=content))
    return messages

```

```

def sanitize_history(history: List[Dict[str, str]]) -> List[Dict[str, str]]:
    normalized: List[Dict[str, str]] = []
    for item in history[-MAX_AGENT_HISTORY:]:
        if not isinstance(item, dict):
            continue
        role = str(item.get("role", "")).strip()

```

```
    if role not in {"user", "assistant"}:
        continue
    content = str(item.get("content", "")).strip()
    if not content:
        continue
    normalized.append({"role": role, "content": content[:MAX_HISTORY_ITEM_LENGTH]})
return normalized
```

```
@lru_cache(maxsize=1)
def build_llm() -> ChatOpenAI:
    api_key = env("DEEPSEEK_API_KEY")
    if not api_key:
        raise RuntimeError("DEEPSEEK_API_KEY 未配置")
    return ChatOpenAI(
        api_key=api_key,
        model=env("AGENT_MODEL", "deepseek-chat"),
        base_url=env("DEEPSEEK_BASE_URL", "https://api.deepseek.com/v1"),
        temperature=0.1,
    )
```

```
def format_status_label(status: str | None) -> str:
    if not status:
        return "未知"
    return STATUS_LABELS.get(status, status)
```

```
def build_empty_result_answer(domain: str, scope: str, next_steps: List[str]) -> str:
    return (
        f"当前未查到{domain}。 \n"
        f"本次查询范围： {scope}。 \n"
        "可能原因： 当前数据中没有符合条件的记录， 或查询关键词过宽/过窄， 或相关业务数据尚未录入。 \n"
        f"建议下一步： {'; '.join(next_steps)}。 "
    )
```

```
def sample_names(rows: List[Dict[str, Any]], key: str, limit: int = 3) -> str:
    values = [str(row.get(key)).strip() for row in rows[:limit] if row.get(key)]
    return "、".join(values)
```

```
def append_tool(state: AgentState, tool_name: str, data: Any) -> None:
    if tool_name in state.get("tools_used", []):
        return
    state.setdefault("tool_outputs", []).append({"tool": tool_name, "data": data})
    state.setdefault("tools_used", []).append(tool_name)
```

```
def append_tool_error(state: AgentState, tool_name: str, error: Exception | str) -> None:
    state.setdefault("tool_errors", []).append({"tool": tool_name, "error": str(error)})
```

```
def safe_append_tool(state: AgentState, tool_name: str, func) -> None:
    try:
        data = func()
        append_tool(state, tool_name, data)
    except Exception as exc:
        append_tool_error(state, tool_name, exc)
        log_debug("tool-error", {"tool": tool_name, "error": str(exc)})
```

```
def has_meaningful_data(value: Any) -> bool:
```

```

if value is None:
    return False
if isinstance(value, list):
    return len(value) > 0
if isinstance(value, dict):
    return any(has_meaningful_data(item) for item in value.values())
return True

```

```

def append_module_context_tools(local_state: AgentState, matched_modules: List[str], message: str) -> None:
    for module_name in matched_modules:
        if module_name == "维修业务":
            keyword = extract_named_keyword(message, ["维修业务", "维修模块", "维修系统", "功能", "模块", "侧边栏", "页面", "包含哪些内容", "包含", "内容", "查看"])
            if is_sidebar_module_question(message) or should_use_broad_query(message) or not keyword:
                keyword = ""
            safe_append_tool(local_state, "query_business_overview", lambda: query_business_overview(keyword=keyword, limit=8))
            safe_append_tool(local_state, "query_orders", lambda: query_orders(keyword="", limit=8))
            safe_append_tool(local_state, "query_repair_personnel", lambda: query_repair_personnel(keyword="", limit=8))
        elif module_name == "进销存":
            safe_append_tool(local_state, "query_inventory", lambda: query_inventory(keyword="", low_stock_only=False, limit=8))
            safe_append_tool(local_state, "query_suppliers", lambda: query_suppliers(keyword="", limit=8))
            safe_append_tool(local_state, "query_supplier_inventory_ranking", lambda: query_supplier_inventory_ranking(limit=5))
        elif module_name == "知识库":
            safe_append_tool(local_state, "query_knowledge_overview", lambda: query_knowledge_overview(keyword="", limit=8))
            safe_append_tool(local_state, "query_rag", lambda: query_rag(question=cleanup_keyword(message) or message, limit=5))
        elif module_name == "基础管理":
            safe_append_tool(local_state, "query_personnel", lambda: query_personnel(keyword="", limit=8))
        elif module_name == "查询统计":
            safe_append_tool(local_state, "query_repair_order_workload", lambda: query_repair_order_workload(message="订单统计 排名", limit=5))
            safe_append_tool(local_state, "query_work_orders", lambda: query_work_orders(keyword="", limit=8))
        elif module_name == "Agent":
            safe_append_tool(local_state, "query_system_features", lambda: query_system_features(message, [module_name]))

```

```

def initialize_state(state: AgentState) -> AgentState:
    return {
        "normalized_message": normalize_message(state["message"]),
        "history": sanitize_history(state.get("history", [])),
        "tool_outputs": [],
        "tools_used": [],
        "tool_errors": [],
    }

```

```

def load_user_state(state: AgentState) -> AgentState:
    user_id = state.get("user_id")
    user_profile = query_user_context(user_id) if user_id else {}
    return {"user_profile": user_profile}

```

```

def analyze_state(state: AgentState) -> AgentState:
    payload = {
        "message": state["normalized_message"],
        "history": state.get("history", []),
    }

```

```

        "user_id": state.get("user_id"),
        "user_profile": state.get("user_profile", {}),
    }
    analysis = json.loads(analyze_user_message.invoke(json.dumps(payload, ensure_ascii=False)))
    log_debug("analysis", analysis)
    return {"analysis": analysis}

def repair_order_node(state: AgentState) -> AgentState:
    message = state["normalized_message"]
    analysis = state["analysis"]
    local_state: AgentState = {"tool_outputs": [], "tools_used": [], "tool_errors": []}

    if analysis.get("needs_workload"):
        safe_append_tool(local_state, "query_repair_order_workload", lambda: query_repair_order_workload(message=message, limit=5))

    if analysis.get("needs_order_detail") and analysis.get("order_id"):
        detail_rows: List[Dict[str, Any]] = []
        try:
            detail_rows = query_repair_order_detail(order_id=analysis["order_id"])
            append_tool(local_state, "query_repair_order_detail", detail_rows)
        except Exception as exc:
            append_tool_error(local_state, "query_repair_order_detail", exc)
            log_debug("tool-error", {"tool": "query_repair_order_detail", "error": str(exc)})
        if not detail_rows:
            safe_append_tool(local_state, "query_orders_by_reference", lambda: query_orders(keyword=str(analysis["order_id"]), limit=10))

    order_keyword = analysis.get("status_keyword") or extract_order_search_keyword(message)
    if (
        analysis.get("use_broad_query")
        or analysis.get("miniprogram_question")
        or is_general_order_lookup(message)
        or ("repair 数据库" in message and not analysis.get("order_id"))
    ) and not analysis.get("status_keyword"):
        order_keyword = ""
    safe_append_tool(local_state, "query_orders", lambda: query_orders(keyword=order_keyword, limit=10))

    if analysis.get("needs_progress") and analysis.get("order_id"):
        safe_append_tool(local_state, "query_progress", lambda: query_progress(order_id=analysis["order_id"]))

    if is_cmms_work_order_query(message) and analysis.get("db_preference") != "repair":
        work_order_keyword = extract_named_keyword(message, ["工单", "维修工单", "派单"])
        if analysis.get("use_broad_query"):
            work_order_keyword = ""
        safe_append_tool(local_state, "query_work_orders", lambda: query_work_orders(keyword=work_order_keyword, limit=8))

    if analysis.get("needs_rag"):
        safe_append_tool(local_state, "query_rag", lambda: query_rag(question=cleanup_keyword(message), limit=5))

    return local_state

def inventory_node(state: AgentState) -> AgentState:
    message = state["normalized_message"]
    analysis = state["analysis"]
    local_state: AgentState = {"tool_outputs": [], "tools_used": [], "tool_errors": []}

    keyword = analysis.get("inventory_keyword", "")
    if analysis.get("use_broad_query"):

```

```

        keyword = ""
        safe_append_tool(
            local_state,
            "query_inventory",
            lambda: query_inventory(keyword=keyword, low_stock_only="低库存" in message, limit=10),
        )

    if "供应商" in message or "供货商" in message:
        supplier_keyword = extract_named_keyword(message, ["供应商", "供货商", "库存金额", "金额最高", "最高", "最近"])
        if analysis.get("use_broad_query"):
            supplier_keyword = ""
        safe_append_tool(local_state, "query_suppliers", lambda: query_suppliers(keyword=supplier_keyword, limit=10))

    if analysis.get("needs_supplier_ranking"):
        safe_append_tool(local_state, "query_supplier_inventory_ranking", lambda: query_supplier_inventory_ranking(limit=5))

    return local_state

def personnel_node(state: AgentState) -> AgentState:
    message = state["normalized_message"]
    analysis = state["analysis"]
    local_state: AgentState = {"tool_outputs": [], "tools_used": [], "tool_errors": []}

    keyword = analysis.get("personnel_keyword", "")
    if analysis.get("use_broad_query") or analysis.get("miniprogram_question"):
        keyword = ""
    if "维修人员" in message or "维修工程师" in message or "维修负责人" in message:
        safe_append_tool(local_state, "query_repair_personnel", lambda: query_repair_personnel(keyword=keyword, limit=10))

    if analysis.get("db_preference") != "repair":
        safe_append_tool(local_state, "query_personnel", lambda: query_personnel(keyword=keyword, limit=10))

    if is_repair_order_analytics_query(message):
        safe_append_tool(local_state, "query_repair_order_workload", lambda: query_repair_order_workload(message=message, limit=5))

    return local_state

def knowledge_node(state: AgentState) -> AgentState:
    message = state["normalized_message"]
    analysis = state["analysis"]
    local_state: AgentState = {"tool_outputs": [], "tools_used": [], "tool_errors": []}

    keyword = analysis.get("knowledge_keyword", "")
    if analysis.get("use_broad_query"):
        keyword = ""
    safe_append_tool(local_state, "query_knowledge_overview", lambda: query_knowledge_overview(keyword=keyword, limit=10))
    safe_append_tool(local_state, "query_rag", lambda: query_rag(question=cleanup_keyword(message) or message, limit=5))

    if analysis.get("matched_modules"):
        safe_append_tool(local_state, "query_system_features", lambda: query_system_features(message, analysis["matched_modules"]))
        append_module_context_tools(local_state, analysis["matched_modules"], message)

    return local_state

```

```

def overview_node(state: AgentState) -> AgentState:
    message = state["normalized_message"]
    analysis = state["analysis"]
    local_state: AgentState = {"tool_outputs": [], "tools_used": [], "tool_errors": []}

    matched_modules = analysis.get("matched_modules", [])
    safe_append_tool(local_state, "query_system_features", lambda: query_system_features(message, matched_modules))
    if matched_modules:
        append_module_context_tools(local_state, matched_modules, message)
    else:
        if any(token in message for token in ["维修业务", "维修模块", "维修系统", "维修功能", "合同", "报告", "机械"]):
            keyword = extract_named_keyword(message, ["维修业务", "维修模块", "维修系统", "里面", "有什么", "功能", "包含", "内容", "查看"])
            if analysis.get("sidebar_question") or analysis.get("use_broad_query") or not keyword:
                keyword = ""
            safe_append_tool(local_state, "query_business_overview", lambda: query_business_overview(keyword=keyword, limit=10))
            if any(token in message for token in ["知识库", "流程", "文档"]):
                safe_append_tool(local_state, "query_knowledge_overview", lambda: query_knowledge_overview(keyword="", limit=5))
                safe_append_tool(local_state, "query_rag", lambda: query_rag(question=cleanup_keyword(message) or message, limit=5))
            return local_state

def general_node(state: AgentState) -> AgentState:
    message = state["normalized_message"]
    analysis = state["analysis"]
    local_state: AgentState = {"tool_outputs": [], "tools_used": [], "tool_errors": []}
    matched_modules = analysis.get("matched_modules", [])
    safe_append_tool(local_state, "query_system_features", lambda: query_system_features(message, matched_modules))
    if matched_modules:
        append_module_context_tools(local_state, matched_modules, message)
    if any(token in message for token in ["订单", "维修单", "进度"]):
        safe_append_tool(local_state, "query_orders", lambda: query_orders(keyword="", limit=8))
    if any(token in message for token in ["人员", "员工", "工程师"]):
        safe_append_tool(local_state, "query_personnel", lambda: query_personnel(keyword="", limit=8))
        safe_append_tool(local_state, "query_repair_personnel", lambda: query_repair_personnel(keyword="", limit=8))
    if any(token in message for token in ["库存", "配件", "供应商"]):
        safe_append_tool(local_state, "query_inventory", lambda: query_inventory(keyword="", low_stock_only="低库存" in message, limit=8))
        safe_append_tool(local_state, "query_suppliers", lambda: query_suppliers(keyword="", limit=8))
        safe_append_tool(local_state, "query_knowledge_overview", lambda: query_knowledge_overview(keyword="", limit=5))
        safe_append_tool(local_state, "query_rag", lambda: query_rag(question=cleanup_keyword(message) or message, limit=5))
    return local_state

def route_scenario(state: AgentState) -> Literal["repair_order", "inventory", "personnel", "knowledge", "overview", "general"]:
    scenario = state["analysis"]["primary_scenario"]
    if scenario not in {"repair_order", "work_order", "inventory", "personnel", "knowledge", "overview"}:
        return "general"
    if scenario == "work_order":
        return "repair_order"
    return scenario

```

```

def merge_branch_results(state: AgentState) -> AgentState:
    return state

def route_post_query(state: AgentState) -> Literal["no_data_node", "answer_node"]:
    tool_outputs = state.get("tool_outputs", [])
    tool_errors = state.get("tool_errors", [])
    if tool_errors and not tool_outputs:
        return "no_data_node"
    if not tool_outputs:
        return "no_data_node"
    if not any(has_meaningful_data(item.get("data")) for item in tool_outputs):
        return "no_data_node"
    return "answer_node"

def no_data_node(state: AgentState) -> AgentState:
    analysis = state.get("analysis", {})
    scenario = analysis.get("primary_scenario", "general")
    tool_errors = state.get("tool_errors", [])
    if tool_errors:
        error_tools = "; ".join(f"{item.get('tool')}: {item.get('error')}" for item in tool_errors[:3])
        return {
            "answer": (
                "本次查询没有返回可用结果，系统已自动进入无数据处理。\\n"
                f"当前场景：{scenario}。\\n"
                f"异常信息：{error_tools}。\\n"
                "建议下一步：请缩小查询范围、补充更精确的订单号/人员名/模块名，或确认相关业务数据是否已同步入库。"
            )
        }

    no_data_messages = {
        "repair_order": "当前未查到符合条件的维修订单或订单进度。建议提供准确订单号，或先查看当前订单列表。",
        "inventory": "当前未查到符合条件的库存或供应商数据。建议先查看全部库存、低库存列表或供应商排行。",
        "personnel": "当前未查到符合条件的人员数据。建议提供人员姓名、手机号或角色关键词继续查询。",
        "knowledge": "当前未查到相关知识库资料。建议补充更具体的流程、制度或文档关键词。",
        "overview": "当前已识别到模块查询，但没有拿到可展示的业务数据。建议继续查看某个模块的明细页面或关联数据。",
        "general": "当前没有查到与该问题直接相关的数据。建议明确查询对象，例如订单、人员、库存、知识库或系统模块。"
    }

    return {"answer": no_data_messages.get(scenario, no_data_messages["general"])}

def build_direct_answer(message: str, tool_outputs: List[Dict[str, Any]]) -> str | None:
    tool_map = {item["tool"]: item.get("data") for item in tool_outputs if item.get("tool")}

    if (
        "query_system_features" in tool_map
        and "query_business_overview" in tool_map
        and any(token in message for token in ["维修业务", "维修模块", "侧边栏"])
    ):
        feature_map = tool_map["query_system_features"] or {}
        business_data = tool_map["query_business_overview"] or {}
        module_features = feature_map.get("维修业务", []) if isinstance(feature_map, dict) else []
        category_rows = business_data.get("repair_categories", []) if isinstance(business_data, dict) else []
        machine_rows = business_data.get("repair_machines", []) if isinstance(business_data, dict) else []
        contract_rows = business_data.get("repair_contracts", []) if isinstance(business_data, dict) else []
        report_rows = business_data.get("repair_reports", []) if isinstance(business_data, dict) else []
        feature_text = ", ".join(module_features[:8]) if module_features else "机械种类管理、机械名称管理、订单管理、检测报告、维修报告、合同管理、维修提醒、维修进度"
        return (

```

```

        "结论：系统侧边栏中的维修业务模块既有页面能力，也已经关联到实际业务数据。\\n"
        f"关键依据：该模块当前包含 {feature_text}。"
        f"同时查到机械分类 {len(category_rows)} 条、机械档案 {len(machine_rows)} 条、维修合同 {len(contract_rows)} 条、维修报告 {len(report_rows)} 条。"
        f"{ '例如机械分类有：' + sample_names(category_rows, 'name') + '。' if category_rows else ''}"
        f"{ '例如机械档案有：' + sample_names(machine_rows, 'name') + '。' if machine_rows else ''}"
        f"{ '例如合同编号有：' + sample_names(contract_rows, 'contract_number') + '。' if contract_rows else ''}"
        f"{ '例如维修报告有：' + sample_names(report_rows, 'report_number') + '。' if report_rows else ''}\\n"
        "下一步建议：你可以继续让我查看维修订单列表、机械档案明细、维修合同列表，或某一张维修报告的详情。"
    )

    if "query_orders" in tool_map and any(token in message for token in ["订单", "维修订单", "小程序订单", "查看订单"]):
        rows = tool_map["query_orders"] or []
        if rows:
            parts = []
            for row in rows[:5]:
                label = format_status_label(row.get("repair_status"))
                parts.append(
                    f"{row.get('order_no') or row.get('id')}(状态{label}, 设备{row.get('machine_name') or '未知'})"
                )
            return f"当前查到的维修订单有：{'；'.join(parts)}。"
        return build_empty_result_answer(
            "维修订单信息",
            "repair 数据库中的 orders 订单列表",
            ["改为指定订单号查询", "补充客户名称、设备名称或订单状态", "继续追问某张订单的负责人或进度"],
        )

    if "query_repair_personnel" in tool_map and any(token in message for token in ["维修人员", "维修工程师", "维修负责人"]):
        rows = tool_map["query_repair_personnel"] or []
        if rows:
            parts = [f"{row.get('name')}(角色{row.get('role') or '未知'}, 电话{row.get('phone') or '暂无'})" for row in rows[:5]]
            return f"当前查到的维修人员有：{'；'.join(parts)}。"
        return build_empty_result_answer(
            "维修人员信息",
            "repair 数据库中的 users 维修管理用户",
            ["改为查询具体姓名或手机号", "确认维修人员是否已在小程序用户库中创建", "继续查询某张订单的负责人"],
        )

    if "query_personnel" in tool_map and any(token in message for token in ["人员", "员工", "部门"]):
        rows = tool_map["query_personnel"] or []
        if rows:
            parts = [f"{row.get('name')}(角色{row.get('role') or '未知'}, 电话{row.get('phone') or '暂无'})" for row in rows[:5]]
            return f"当前查到的人员有：{'；'.join(parts)}。"

    if "query_suppliers" in tool_map and "供应商" in message and "最高" not in message and "金额" not in message:
        rows = tool_map["query_suppliers"] or []
        if rows:
            parts = [f"{row.get('name')}(编码{row.get('code') or '暂无'}, 电话{row.get('phone') or '暂无'})" for row in rows[:5]]
            return f"当前查到的供应商有：{'；'.join(parts)}。"
        return build_empty_result_answer(
            "供应商信息",
            "repair 数据库中的 suppliers 供应商档案",
            ["改为查询具体供应商名称", "查看库存金额最高的供应商排行", "确认供应商资料是否已建档"],
        )

    if "query_work_orders" in tool_map and any(token in message for token in ["工单", "派单", "故障工单"]):
        rows = tool_map["query_work_orders"] or []
        if rows:

```

```

        parts = [f"{row.get('order_no')}(状态{row.get('status') or '未知'}, 故障{row.get('fault_type') or '未填写'})" for r
ow in rows[:5]]
        return f"当前查到的系统工单有: {'; '.join(parts)}。"
    return build_empty_result_answer(
        "系统工单",
        "cmms_db 数据库中的 work_orders 工单列表",
        ["补充工单号或故障类型", "改问维修订单而不是系统工单", "确认工单是否已进入 CMMS 工单库"],
    )

    if "query_business_overview" in tool_map and any(token in message for token in ["维修业务", "维修模块", "维修
系统"]):
        data = tool_map["query_business_overview"] or {}
        if isinstance(data, dict):
            category_rows = data.get("repair_categories", []) or []
            machine_rows = data.get("repair_machines", []) or []
            contract_rows = data.get("repair_contracts", []) or []
            report_rows = data.get("repair_reports", []) or []
            categories = len(category_rows)
            machines = len(machine_rows)
            contracts = len(contract_rows)
            reports = len(report_rows)
            if categories == 0 and machines == 0 and contracts == 0 and reports == 0:
                return (
                    "维修业务模块已识别成功, 但当前这次按业务概览条件未命中明细数据。"
                    "这通常意味着问句中的描述被当成筛选词后没有匹配到记录, 并不等于模块本身没有数据。"
                    "建议继续直接查看维修业务模块包含的机械分类、机械档案、维修合同或维修报告明细。"
                )
            category_examples = sample_names(category_rows, "name")
            machine_examples = sample_names(machine_rows, "name")
            contract_examples = sample_names(contract_rows, "contract_number")
            report_examples = sample_names(report_rows, "report_number")
            return (
                "结论: 维修业务模块已经关联到完整的业务数据面, 可用于查看机械分类、机械档案、维修合同和维修报告。\\n
"
                f"关键依据: 当前查到机械分类 {categories} 条、机械档案 {machines} 条、维修合同 {contracts} 条、维修报告
{reports} 条。"
                f"{'例如机械分类有: ' + category_examples + '。' if category_examples else ''}"
                f"{'例如机械档案有: ' + machine_examples + '。' if machine_examples else ''}"
                f"{'例如合同编号有: ' + contract_examples + '。' if contract_examples else ''}"
                f"{'例如维修报告有: ' + report_examples + '。' if report_examples else ''}\\n"
                "下一步建议: 你可以继续让我分别查看 “机械分类明细” “机械档案列表” “维修合同列表” 或 “维修报告列表
” 。"
            )

    if "query_inventory" in tool_map and "低库存" in message:
        rows = tool_map["query_inventory"] or []
        if not rows:
            return build_empty_result_answer(
                "低库存配件",
                "repair 数据库中的 spare_parts, 筛选条件为 stock_quantity <= min_stock",
                ["改为查看全部配件库存", "查看某个配件名称或编码", "确认最低库存阈值是否已维护"],
            )
        parts = [f"{row.get('part_name') or row.get('part_code')}(库存{row.get('stock_quantity', 0)}, 最低{row.get('min_
stock', 0)})" for row in rows[:5]]
        return f"当前低库存配件已按紧急程度优先列出: {'; '.join(parts)}。"

    if "query_repair_order_detail" in tool_map:
        rows = tool_map["query_repair_order_detail"] or []
        if rows:
            row = rows[0]
            progress_value = row.get("progress")
            progress_text = f"{progress_value}%" if progress_value is not None else "暂无"
            return (

```

```

        f"订单 {row.get('id')} 当前处于 "{format_status_label(row.get('status'))}" 阶段, "
        f"当前进度 {progress_text}, 最近一次更新是 {row.get('progress_updated_at') or row.get('updated_at') or '
暂无}", "
        f"当前维修人员是 {row.get('assigned_to_name') or '未分配'}. "
    )
    reference_rows = tool_map.get("query_orders_by_reference") or []
    if reference_rows:
        parts = []
        for row in reference_rows[:5]:
            parts.append(f"{row.get('order_no') or row.get('id')}(状态{format_status_label(row.get('repair_status'))})")
        return (
            f"当前未查到 ID 为 {detect_order_id(message) or '该编号'} 的精确维修订单。"
            f"但在 repair 订单库中查到了与该编号相关的候选订单: {'; '.join(parts)}。"
            " 你可以继续指定准确订单号, 或直接让我查看其中某一张订单的状态、进度和负责人。"
        )
    return build_empty_result_answer(
        "该维修订单明细",
        "repair 数据库中的指定订单详情",
        ["确认订单 ID 是否正确", "先查看当前订单列表", "改为按订单号或设备名称查询"],
    )

    if "query_repair_order_workload" in tool_map:
        rows = tool_map["query_repair_order_workload"] or []
        if rows:
            top = rows[0]
            ranking = "; ".join(f"{row.get('technician_name', '未知人员')} {row.get('order_count', 0)} 单" for row in rows[
:5])
            return f"当前订单量最高的人员是 {top.get('technician_name', '未知人员')}, 共 {top.get('order_count', 0)} 单。
排行参考: {ranking}。"

    if "query_supplier_inventory_ranking" in tool_map:
        rows = tool_map["query_supplier_inventory_ranking"] or []
        if rows:
            parts = [f"{row.get('name')}(库存金额{float(row.get('inventory_value', 0)):.2f}元, 配件{row.get('part_count', 0)
}种)" for row in rows[:5]]
            return f"最近库存金额最高的供应商如下: {'; '.join(parts)}。"

    if "query_system_features" in tool_map and any(token in message for token in ["模块", "菜单", "功能", "入口", "侧
边栏", "页面"]):
        data = tool_map["query_system_features"] or {}
        if data:
            parts = [f"{module}: {'; '.join(features[:6])}" for module, features in list(data.items())[:4]]
            return (
                "结论: 当前问题已命中系统侧边栏模块定义。\\n"
                f"关键依据: {'; '.join(parts)}。\\n"
                "下一步建议: 如果你要看某个模块的真实业务数据, 我可以继续展开对应模块的明细, 例如维修业务、进销存或
知识库。"
            )
        return build_empty_result_answer(
            "系统模块内容",
            "cmms_db 侧的系统模块能力映射",
            ["直接问某个侧边栏模块", "改问维修业务、进销存或知识库模块", "补充页面入口或功能关键词"],
        )

    if "query_knowledge_overview" in tool_map and any(token in message for token in ["知识库", "维修流程"]):
        data = tool_map["query_knowledge_overview"] or {}
        files = data.get("files", []) if isinstance(data, dict) else []
        chunks = data.get("chunks", []) if isinstance(data, dict) else []
        if files or chunks:
            names = [item.get("original_name") for item in files[:3] if item.get("original_name")]
            excerpt = chunks[0].get("excerpt") if chunks else ""
            if names:
                return (

```

```

        "结论：知识库中已经存在与当前问题相关的资料。\\n"
        f"关键依据：相关资料包括 {'、'.join(names)}。"
        f"{' 其中一条摘要为： ' + excerpt if excerpt else ''}\\n"
        "下一步建议：你可以继续让我总结维修流程、解释某一份文档，或者提取某个流程节点的操作要求。"
    )
    return build_empty_result_answer(
        "知识库相关资料",
        "cmms_db 中的 kb_collections、kb_files、kb_chunks",
        ["补充更具体的流程名称", "改问某个模块相关文档", "确认相关文档是否已入库并完成切分"],
    )

return None

def answer_node(state: AgentState) -> AgentState:
    tool_outputs = state.get("tool_outputs", [])
    message = state["message"]
    direct_answer = build_direct_answer(state["normalized_message"], tool_outputs)
    if direct_answer:
        return {"answer": direct_answer}

    llm = build_llm()
    scenario = state["analysis"]["primary_scenario"]
    system_prompt = (
        f"{ENTERPRISE_SYSTEM_PROMPT}\\n"
        f"当前场景要求：{SCENARIO_PROMPTS.get(scenario, SCENARIO_PROMPTS['general'])}"
    )
    prompt = [
        SystemMessage(content=system_prompt),
        *convert_history(state.get("history", [])),
        HumanMessage(
            content=(
                f"当前用户ID：{state.get('user_id')}\\n"
                f"用户上下文：{json.dumps(state.get('user_profile', {}), ensure_ascii=False, default=str)}\\n"
                f"用户原始问题：{message}\\n"
                f"归一化问题：{state['normalized_message']}\\n"
                f"场景分析：{json.dumps(state['analysis'], ensure_ascii=False)}\\n"
                f"数据库查询结果：{json.dumps(tool_outputs, ensure_ascii=False, default=str)}\\n\\n"
                "请基于这些结果直接作答。如果多组数据冲突，优先说明差异，不要自行杜撰。"
            )
        ),
    ]
    response = llm.invoke(prompt)
    answer = response.content if isinstance(response, AIMessage) else str(response)
    return {"answer": answer}

```

```

@lru_cache(maxsize=1)
def build_graph():
    graph = StateGraph(AgentState)
    graph.add_node("initialize_state", initialize_state)
    graph.add_node("load_user_state", load_user_state)
    graph.add_node("analyze_state", analyze_state)
    graph.add_node("repair_order", repair_order_node)
    graph.add_node("inventory", inventory_node)
    graph.add_node("personnel", personnel_node)
    graph.add_node("knowledge", knowledge_node)
    graph.add_node("overview", overview_node)
    graph.add_node("general", general_node)
    graph.add_node("merge_branch_results", merge_branch_results)
    graph.add_node("no_data_node", no_data_node)
    graph.add_node("answer_node", answer_node)

```

```

graph.add_edge(START, "initialize_state")
graph.add_edge("initialize_state", "load_user_state")
graph.add_edge("load_user_state", "analyze_state")
graph.add_conditional_edges(
    "analyze_state",
    route_scenario,
    {
        "repair_order": "repair_order",
        "inventory": "inventory",
        "personnel": "personnel",
        "knowledge": "knowledge",
        "overview": "overview",
        "general": "general",
    },
)
graph.add_edge("repair_order", "merge_branch_results")
graph.add_edge("inventory", "merge_branch_results")
graph.add_edge("personnel", "merge_branch_results")
graph.add_edge("knowledge", "merge_branch_results")
graph.add_edge("overview", "merge_branch_results")
graph.add_edge("general", "merge_branch_results")
graph.add_conditional_edges(
    "merge_branch_results",
    route_post_query,
    {
        "no_data_node": "no_data_node",
        "answer_node": "answer_node",
    },
)
graph.add_edge("no_data_node", END)
graph.add_edge("answer_node", END)
return graph.compile()

```

```

def invoke_agent(message: str, history: List[Dict[str, str]], user_id: int | None) -> Dict[str, Any]:
    graph = build_graph()
    state = graph.invoke(
        {
            "message": (message or "").strip(),
            "history": sanitize_history(history),
            "user_id": user_id,
        }
    )
    return {
        "answer": state.get("answer", ""),
        "tools_used": state.get("tools_used", []),
        "scenario": state.get("analysis", {}).get("primary_scenario", "general"),
        "analysis": state.get("analysis", {}),
    }

```

// frontend-web/src/App.vue

```

<template>
  <router-view v-slot="{ Component }">
    <keep-alive :include="cachedViews">
      <component :is="Component" :key="$route.fullPath" />
    </keep-alive>
  </router-view>
</template>

<script setup>

```

```
import { onMounted } from 'vue'
import { useAuthStore } from '@/stores/auth'
import { useThemeStore } from '@/stores/theme'

const authStore = useAuthStore()
const themeStore = useThemeStore()

onMounted(() => {
  // 初始化主题
  themeStore.initTheme()

  // 尝试从 localStorage 恢复登录状态
  console.log('App 初始化, 从 localStorage 加载认证信息')
  authStore.loadAuthFromStorage()
  console.log('加载后 token:', authStore.token)
  console.log('加载后 userInfo:', authStore.userInfo)
})
</script>

<style lang="scss">
#app {
  width: 100%;
  height: 100vh;
}
</style>
```

// frontend-web/src/api/agent.js

```
import request from './request'

export function sendAgentMessage(data) {
  return request({
    url: '/agent/chat',
    method: 'post',
    data,
    timeout: 180000
  })
}
```

// frontend-web/src/api/agreement.js

```
import request from './request'

/**
 * 获取免责声明
 */
export function getAgreement() {
  return request({
    url: '/agreement',
    method: 'get'
  })
}

/**
 * 获取协议详情
 */
export function getAgreementDetail(id) {
  return request({
    url: `/agreement/${id}`,
    method: 'get'
  })
}
```

```
    })  
  }  
  
  /**  
   * 保存协议  
   */  
  export function saveAgreement(data) {  
    return request({  
      url: '/agreement',  
      method: 'post',  
      data  
    })  
  }  
  
  /**  
   * 更新协议  
   */  
  export function updateAgreement(id, data) {  
    return request({  
      url: `/agreement/${id}`,  
      method: 'put',  
      data  
    })  
  }  
  
  /**  
   * 预览协议  
   */  
  export function previewAgreement(id) {  
    return request({  
      url: `/agreement/${id}/preview`,  
      method: 'get'  
    })  
  }  
}
```

// frontend-web/src/api/auth.js

```
import request from './request'  
  
/**  
 * 用户登录  
 */  
export function login(data) {  
  return request({  
    url: '/auth/login',  
    method: 'post',  
    data  
  })  
}
```